

RICERCA TECNICA — FONTI VERIFICATE

Deployment OpenClaw aziendale: deterministico, verificabile, a prova di invenzione

Che cosa impedisce davvero un'invenzione, che cosa la registra soltanto, e che cosa oggi con OpenClaw non è ottenibile.

VERSIONE ANALIZZATA

OpenClaw 2026.7.1-2 — commit 0790d9f

DATA

19 agosto 2026

INSTALLAZIONE

/Users/apple/.local/lib/node_modules/openclaw

AUTORITÀ SULLE CHIAVI DI CONFIG

schema estratto con `openclaw config schema` — 2.519.129 byte, 8.516 percorsi

Ogni affermazione non contrassegnata come NON VERIFICATO è ancorata a un percorso `file:riga` del pacchetto installato, a una pagina di documentazione presente localmente, o a un output di comando riportato letteralmente nel registro di lavoro.

Indice

Deployment OpenClaw aziendale: deterministico, verificabile, a prova di invenzione	5
Avvertenza preliminare: tre pagine richieste non esistono	5
Sommario esecutivo in cinque righe	5
(a) Risposte operative	6
1. Accesso ai dati: MCP parametrizzato contro plugin tool proprio	6
1.1 Schema completo di mcp.servers — estratto dallo schema reale	6
1.2 I default dei timeout, e la trappola dei 1,5 secondi	7
1.3 Come i tool MCP arrivano al modello	7
1.4 I tre cancelli sono in serie, non in concorrenza	7
1.5 Blocco di config per un DB aziendale	8
1.6 Verdetto: server MCP, con plugin di policy separato solo se serve approvazione	9
1.7 compat.supportsStrictMode: il nome inganna	9
2. llm-task: limiti reali	10
2.1 "Isolated completion": la primitiva non esiste con quel nome	10
2.2 Quali harness lo supportano, e cosa succede quando no	10
2.3 Validazione dello schema: TypeBox, un colpo solo, zero retry	11
2.4 La temperature è onorata? Dipende, e in due casi sparisce in silenzio	11
2.5 Il timeout è mascherato da errore di formato	11
2.6 Blocco di config	11
3. Lobster: pipeline in una chiamata, e i suoi limiti veri	12
3.1 Non è installato qui	12
3.2 Cosa promette, e come si abilita	12
3.3 Come si innesta llm-task — e il limite documentato è diverso da quello atteso	12
3.4 L'ipotesi sull'exec è corretta, e la prova sta nel codice	13
3.5 Il resume token non è firmato e non scade	13
3.6 Quando conviene	13
4. Il cancello anti-invenzione	13
4.1 Catalogo hook: 40 nomi, e uno solo blocca davvero l'uscita	14
4.2 before_agent_finalize è fail-open in sei modi. Nessuno è un bug: è il progetto.	14
4.3 message_sending è l'unico blocco duro	15
4.4 before_agent_reply: funziona, ma non serve a questo	15
4.5 allowConversationAccess: il percorso della richiesta è sbagliato	16
4.6 Come accede l'hook alle prove raccolte dai tool	16
4.7 L'architettura del cancello: tre stadi, non uno	17
4.8 Esiste già: llm-grounded su ClawHub — conviene adottarlo	17
5. Latenza: dove si perdono i secondi su una domanda banale	18
5.1 Prima di tutto, la misura	18

5.2 Il thinking: un solo interruttore reale su DeepSeek V4	19
5.3 Checklist, ordinata per impatto, con le chiavi verificate	19
5.4 Tre avvertenze controintuitive	20
6. Determinismo del modello	20
6.1 La temperature non ha una chiave: viaggia in un contenitore free-form	20
6.2 compat.supportsTemperature è un flag morto	21
6.3 --raw-stream esiste, ma non prova quello che serve	21
6.4 La procedura che invece funziona	21
6.5 DeepSeek V4 con thinking	22
7. Isolamento multiutente	22
7.1 La risposta scomoda, in una citazione	23
7.2 I tre controlli, e cosa garantisce ciascuno	23
7.3 Sandbox: cosa isola, e i suoi buchi dichiarati	23
7.4 toolsBySender: filtra in uscita, non ri-aggiunge	24
7.5 workspaceOnly per agente può ALLENARE il globale	24
7.6 Exec: le modalità, e come si spegne davvero	24
7.7 Threat model: i rischi residui dichiarati	25
7.8 Stato reale di questa macchina	25
8. Verificabilità: cosa resta registrato	25
8.1 Chiarimento che evita un errore da contratto	26
8.2 Il ledger: cosa contiene, cosa no, e quanto dura	26
8.3 Le sei superfici, e dove stanno	26
8.4 Trajectory: il più completo, e il più fragile	27
8.5 OpenTelemetry: captureContent è l'unica via per esportare il contenuto	27
8.6 Prometheus: la chiave non esiste, ed è corretto	28
8.7 Il registro append-only che si può davvero costruire	28
9. Esiste di meglio?	29
9.1 Dentro OpenClaw	29
9.2 Fuori da OpenClaw	30
9.3 Verdetto	30
9.4 Architettura ibrida raccomandata	31
10. Rischi: cosa può rompersi al prossimo aggiornamento	32
10.1 Nota di metodo	32
10.2 Il gate d'uscita non è garantito — blocker assoluto	32
10.3 La premessa «le prove dei tool sono reali» non regge	32
10.4 Deprecazioni con data — due sono già scadute	33
10.5 MCP: rottura annunciata	33
10.6 Sandbox: il verde mente, con label P1	33
10.7 Identità e attribuzione	34
10.8 Il CHANGELOG non aiuta	34
10.9 Le tre cose da mettere in contratto	34

(b) Quale livello impedisce quale errore **35**

(c) Cosa con OpenClaw oggi NON è ottenibile **37**

(d) Registro di lavoro **38**

 d.1 Metodo, e un incidente da dichiarare 38

 d.2 Comandi eseguiti, con output grezzo 38

 d.3 Pagine consultate, una per una 41

 d.4 Ricerche che non hanno dato risultato 41

 d.5 Cosa è stato verificato e cosa è stato solo letto 42

(e) NON VERIFICATO **43**

 Su OpenClaw 43

 Sulle issue GitHub 44

 Sui plugin di terze parti 44

 Sui framework alternativi 45

 Sul metodo di questo documento 45

Deployment OpenClaw aziendale: deterministico, verificabile, a prova di invenzione

Versione analizzata: OpenClaw 2026.7.1-2 (commit 0790d9f) **Data:** 19 agosto 2026 **Installazione:** /Users/apple/.local/lib/node_modules/openclaw **Metodo:** ogni affermazione è ancorata a file:riga del pacchetto installato o a una pagina docs.openclaw.ai presente localmente. Lo schema di configurazione è stato estratto con openclaw config schema (2.519.129 byte, 8.516 percorsi) e usato come unica autorità sull'esistenza delle chiavi.

Avvertenza preliminare: tre pagine richieste non esistono

Delle 32 pagine indicate, tre non esistono in questa versione. Verificato sul filesystem e sui redirect dichiarati in docs/docs.json (238 redirect totali):

Pagina richiesta	Stato in 2026.7.1-2	Pagina reale
/tools/mcp	non esiste	/cli/mcp (redirect dichiarato solo da /mcp)
/gateway/audit	non esiste, nessun redirect	/cli/audit + /gateway/security/audit-checks
/gateway/multi-tenant-hosting	non esiste, nessun redirect	vedi punto 7: la multi-tenancy è dichiarata fuori scopo

Le altre 29 esistono. Inoltre docs/tools/code-execution.md **non** documenta tools.codeMode: descrive il tool code_execution del plugin xAI (analisi Python remota su grok-4-1-fast). Il documento giusto per code mode è docs/reference/code-mode.md.

Sommario esecutivo in cinque righe

1. Il livello che il committente crede sia il cancello anti-invenzione — before_agent_finalize — è **fail-open per costruzione, in sei modi distinti**. Non blocca: chiede una revisione, e se qualcosa va storto lascia passare la risposta originale.
2. L'unico blocco duro sull'uscita è message_sending con {cancel:true}, dichiarato terminale.
3. Esiste già un plugin pubblico che implementa esattamente questa architettura (llm-grounded su ClawHub): conviene studiarlo prima di riscriverlo.
4. Per l'accesso ai dati, il server MCP con operazioni parametrizzate batte il plugin tool proprio, e la ragione decisiva è l'isolamento di processo.
5. OpenClaw si autodichiara **68% — Alpha** (docs/maturity/scorecard.md:22-32) e dichiara di **non** essere un confine di sicurezza multi-tenant. Entrambe le cose vanno dette al cliente prima del contratto.

(a) Risposte operative

1. Accesso ai dati: MCP parametrizzato contro plugin tool proprio

1.1 Schema completo di mcp.servers — estratto dallo schema reale

Verificato con openclaw config schema. Questi e **solo** questi sono i campi ammessi in 2026.7.1-2:

Campo	Tipo	Note
enabled	boolean	
command	string	obbligatorio se transport: "stdio" (superRefine, dist/zod-schema-09ml_nmo.js:943-946)
args	array	
env	object	filtrato dalla host environment security policy (docs/cli/mcp.md:653-658)
cwd / workingDirectory	string	alias
url	string	per trasporti HTTP
transport	enum	stdio sse streamable-http (dist/zod-schema-09ml_nmo.js:896-901)
headers	object	ignorato se auth: "oauth" (docs/cli/mcp.md:700)
connectionTimeoutMs	number	ms
connectTimeout / connect_timeout	number	secondi , moltiplicati ×1000
requestTimeoutMs	number	ms
timeout	number	secondi
supportsParallelToolCalls / supports_parallel_tool_calls	boolean	se true -> executionMode: "parallel" (dist/agent-bundle-mcp-materialize-D9l-gQ5S.js:150)
auth	string	letterale: solo "oauth" (dist/zod-schema-09ml_nmo.js:913)
oauth	object	solo scope, redirectUrl, clientMetadataUrl
sslVerify / ssl_verify	boolean	
clientCert / client_cert, clientKey / client_key	string	
toolFilter	object	include[], exclude[], glob solo *
codex	object	agents[], defaultToolsApprovalMode, default_tools_approval_mode — solo per thread Codex app-server

Più mcp.sessionIdleTtlMs (default 600.000 ms, dist/schema-DRy01XBt.js:933).

Correzione a una premessa della richiesta. oauth.identity **non esiste**. Il blocco oauth è .strict() con additionalProperties: false:

```
"oauth": {
  "type": "object",
  "properties": {
    "scope": { "type": "string", "minLength": 1 },
    "redirectUrl": { "type": "string", "format": "uri" },
    "clientMetadataUrl": { "type": "string", "format": "uri" }
  },
  "additionalProperties": false,
  "title": "MCP OAuth"
}
```

Scrivere `oauth.identity` in `openclaw.json` non è un campo ignorato: è un errore di validazione.

Allo stesso modo `auth` non è una stringa libera. Nel codice:

```
// dist/zod-schema-09ml_nmo.js:913
auth: literal("oauth").optional(),
```

1.2 I default dei timeout, e la trappola dei 1,5 secondi

```
// dist/agent-bundle-mcp-runtime--G82BMs.js:546-547
const DEFAULT_CONNECTION_TIMEOUT_MS = 3e4; // 30 s
const DEFAULT_REQUEST_TIMEOUT_MS = 6e4; // 60 s
```

Confermato dal vivo — `openclaw mcp status --verbose`, estratto letterale:

```
- meta-dev: disabled oauth
  launch: https://mcp.facebook.com/devtools
  timeouts: connect=30000ms request=60000ms
  oauth: tokens=no client=no
- sqlite: disabled
  launch: node /Users/apple/.openclaw/workspace/mcp-servers/mcp-sqlite.mjs
  timeouts: connect=30000ms request=60000ms
```

La trappola sta nell'enumerazione del catalogo, che avviene **prima** del primo token di ogni turno:

```
// dist/agent-bundle-mcp-runtime--G82BMs.js:765, 909-912
const BUNDLE_MCP_CATALOG_LIST_TIMEOUT_MS = 1500;
function getCatalogListTimeoutMs(rawServer, requestTimeoutMs) {
  if (bundleMcpCatalogListTimeoutMs !== void 0) return bundleMcpCatalogListTimeoutMs;
  return hasConfiguredMcpRequestTimeout(rawServer) ? requestTimeoutMs :
    BUNDLE_MCP_CATALOG_LIST_TIMEOUT_MS;
}
```

Se **non** configuri `requestTimeoutMs`, il `tools/list` è tagliato a 1,5 s: protezione buona. Se lo configuri a 60000 "per sicurezza", quel tetto salta e il `tools/list` eredita il tuo valore — **peggiori la latenza di avvio turno di 40×**.

Concorrenza di connessione: 6 server in parallelo (`BUNDLE_MCP_CATALOG_CONNECT_CONCURRENCY = 6, :767`).

1.3 Come i tool MCP arrivano al modello

Nome model-facing: `<safeServerName>__<toolName>`, doppio underscore

(`dist/agent-bundle-mcp-names-B9PLR-i.js:39`). Sanificazione (:4-24): caratteri fuori da `[A-Za-z0-9_-]` sostituiti con `-`; prefisso server max 30 caratteri, nome totale max 64; collisioni risolte con suffisso `-2`, `-3`.

Due protezioni che il percorso MCP dà e il plugin tool no:

- I risultati portano `untrustedMcpOutput: true` nei details (`dist/agent-bundle-mcp-materialize-D9l-gQ5S.js:84`).
- I metadati sono ripuliti da prompt injection: `sanitizeMcpMetadataText` neutralizza `ignore (all) previous instructions, disregard ... instructions`, e normalizza system prompt (`dist/agent-bundle-mcp-runtime--G82BMs.js:959-963`).

Su un database aziendale con testo di provenienza mista, questa seconda protezione non è un dettaglio.

1.4 I tre cancelli sono in serie, non in concorrenza

Non "vince" nessuno: basta che uno chiuda.

Livello 1 — `toolFilter`, applicato al nome **grezzo**, in fase di discovery:

```
// dist/agent-bundle-mcp-runtime--G82BMQs.js:953-958
function shouldExposeMcpTool(selection, toolName) {
  const include = selection.include ?? [];
  const exclude = selection.exclude ?? [];
  if (include.length > 0 && !include.some((p) => matchesMcpToolFilterPattern(p, toolName))) return false;
  return !exclude.some((p) => matchesMcpToolFilterPattern(p, toolName));
}
```

`exclude` batte `include`. Glob solo `*`, nessuna regex (`dist/agent-bundle-mcp-filter-DA8pW_vZ.js:3-23`).

Livello 2 — `tools.allow` / `tools.deny`, applicato al nome **prefissato**, oppure a `bundle-mcp`, oppure a `group:plugins`, oppure al namespace del server:

```
// dist/tool-policy-pipeline-C3ed0W1F.js:24-33
function denylistBlocksMcpServer(params) {
  return denylistBlocksName("bundle-mcp", params.denylist)
    || matchesAnyGlobPattern("group:plugins", params.denylist)
    || denylistBlocksMcpServerNamespace({...});
}
```

Livello 3 — `tools.sandbox.tools`, solo su sessioni sandboxed, cancello **aggiuntivo**. Citazione diretta (`docs/gateway/sandbox-vs-tool-policy-vs-elevated.md:110`):

«For sandboxed MCP servers, the sandbox tool policy is a second allow gate. If `mcp.servers` is configured but `sandboxed` turns only show built-in tools, add `bundle-mcp`, `group:plugins`, or a server-prefixed MCP tool name/glob such as `outlook__send_mail` or `outlook__*` to `tools.sandbox.tools.alsoAllow`»

Guasto silenzioso da mettere in checklist: senza quella voce il server **si connette**, `mcp` status è verde, probe funziona, e i tool **non arrivano al modello** (`docs/gateway/config-tools.md:79`). E `openclaw doctor` copre solo i server in `mcp.servers`, non quelli da manifest di plugin o da `.mcp.json` (`docs/gateway/sandbox-vs-tool-policy-vs-elevated.md:112`).

1.5 Blocco di config per un DB aziendale

Il modello non scrive mai SQL: chiama operazioni parametrizzate.

```
{
  "mcp": {
    "servers": {
      "dbaziendale": {
        "enabled": true,
        "transport": "stdio",
        "command": "node",
        "args": ["/opt/mcp/db-aziendale/server.mjs"],
        "env": { "DATABASE_URL": "postgres://readonly@..." },
        "connectionTimeoutMs": 5000,
        "requestTimeoutMs": 30000,
        "supportsParallelToolCalls": false,
        "toolFilter": {
          "include": ["fatture_*", "clienti_leggi", "ordini_per_periodo"]
        }
      }
    }
  },
  "tools": {
    "profile": "minimal",
    "alsoAllow": ["bundle-mcp"],
    "deny": ["exec", "process", "write", "edit", "apply_patch", "group:fs"]
  }
}
```

Note su ogni riga che conta:

- `requestTimeoutMs: 30000` è una scelta consapevole: alza il tetto del `tools/list` da 1,5 s a 30 s. Se il server enumera in fretta va bene; se no, togliilo e lascia il default.
- `toolFilter.include` è il secondo lucchetto: se domani qualcuno aggiunge `run_sql` al server, non entra nel catalogo — e si cambia in **config**, senza rebuild.
- `profile: "minimal"` espone solo `session_status`; `alsoAllow: ["bundle-mcp"]` riapre i soli tool MCP.

- Il `deny` esplicito è ridondante con `minimal`, ma sopravvive a un cambio di profilo fatto da qualcun altro.

1.6 Verdetto: server MCP, con plugin di policy separato solo se serve approvazione

Dimensione	Server MCP (stdio)	Plugin tool proprio
Isolamento processo	Processo figlio, ucciso come process tree (docs/cli/mcp.md:82). Un crash non abbatte il Gateway	Nessuno : importato nel processo OpenClaw (docs/plugins/tool-plugins.md:231)
Circuit breaker	Fallimenti ripetuti mettono in pausa il server (docs/cli/mcp.md:378)	Assente
Output marcato untrusted	Sì (materialize-D9l-gQ5S.js:84) + anti-injection sui metadati	No
<code>toolFilter</code>	Sì, in config, senza rebuild	No
Timeout dichiarativi	Sì, per server	No: solo <code>context.signal?.throwIfAborted()</code>
Versionamento	Debole : nessun campo versione nello schema	Forte : <code>contracts.tools + plugins build --check</code> in CI
<code>before_tool_call</code> / approvazioni	Funziona (i tool MCP sono <code>pluginId: "bundle-mcp"</code>)	Funziona, e può stare nello stesso pacchetto

Scegli il server MCP. La ragione decisiva è la sola che non puoi ricreare: un driver DB in-process significa che un pool esaurito o un `unhandledRejection` colpisce tutti gli agenti.

Compensa l'unico punto dove il plugin vince — il versionamento — mettendo in pipeline `openclaw mcp doctor --probe` e confrontando l'array `tools[]` di `probe --json` con una lista attesa: se cambia, il build fallisce. È un banco che misura, non un elenco scritto a mano.

Se serve approvazione umana sulle scritture, quella logica va in un plugin separato che registra `before_tool_call` con `requireApproval`, o meglio `api.registerTrustedToolPolicy(...)`, che gira **prima** degli hook ordinari e richiede che ogni id sia dichiarato in `contracts.trustedToolPolicies` (docs/plugins/hooks.md:262–275).

1.7 `compat.supportsStrictMode`: il nome inganna

La chiave esiste, in un solo punto: `models.providers.<id>.models[].compat.supportsStrictMode`. Ma **non produce** `strict: true`:

```
// dist/stream-BcRkg2P0.js:39-43
function resolveOpenAIStrictToolSetting(model, options) {
  if (resolvesToNativeOpenAIStrictTools(model, options?.transport ?? "stream")) return true;
  if (options?.supportsStrictMode) return false;
}
```

`resolvesToNativeOpenAIStrictTools` (:26–38) ritorna `true` **solo** se la rotta è nativa OpenAI e il provider è `openai`, `azure-openai` o `azure-openai-responses`. Le tre uscite reali:

Condizione	Cosa finisce nel payload
Rotta nativa OpenAI/Azure	<code>strict: true</code> — poi soggetto a downgrade
Non-OpenAI e <code>supportsStrictMode: true</code>	<code>strict: false</code> esplicito
Non-OpenAI e flag assente/false	campo <code>strict</code> omesso

E anche con `strict: true` c'è un downgrade automatico se un solo schema non è compatibile (node_modules/@openclaw/ai/dist/openai-responses-shared-sj2YUPYc.mjs:1182-1185): bastano un `anyOf`, un `type` come `array`, un `type: "object"` senza `additionalProperties: false`, o una `property` non elencata in `required`. Il downgrade è loggato solo a livello `debug`: **in condizioni normali non lo vedi**.

Prova grezza su quali modelli lo dichiarano:

```
$ grep -rn '"supportsStrictMode": *true|supportsStrictMode: *true' \
  /Users/apple/.local/lib/node_modules/openclaw /Users/apple/.openclaw
(nessun risultato)
```

Tutte le occorrenze sono false (1 in `dist/extensions/opencode/openclaw.plugin.json:173`, 28 in `~/.openclaw/openclaw.json`).

Conseguenza operativa: su un modello non-OpenAI non puoi contare sullo strict schema enforcement lato provider per impedire al modello di inventare parametri. La validazione dei parametri va fatta nel server MCP.

Sull'output strutturato in generale: OpenClaw non ha un'astrazione propria. `response_format` è un extra-param passthrough, request-scoped (`dist/extra-params-cce1g0up.js:77-78`), tradotto per la Responses API (`dist/openai-transport-stream-B0WkSqXp.js:2104-2109`), e **non validato**: se il provider lo ignora, OpenClaw non se ne accorge.

2. `llm-task`: limiti reali

Stato su questa installazione — output letterale di `openclaw plugins list`:

LLM Task	llm-task	openclaw	disabled	stock:llm-task/index.js
2026.7.1				Generic JSON-only LLM tool for structured tasks
				callable from workflows.

Presente, **disabilitato**. File unico: `dist/extensions/llm-task/index.js`, 229 righe.

2.1 "Isolated completion": la primitiva non esiste con quel nome

Ricerca di `isolated` / `isolatedCompletion` in `dist/extensions/llm-task/index.js` e `dist/embedded-agent-*.js`: **zero occorrenze**. Quello che esiste è `runEmbeddedAgent` con i tool spenti:

```
// dist/extensions/llm-task/index.js:154, 169
api.runtime.agent.runEmbeddedAgent({ ... disableTools: true ... })
```

L'isolamento è parziale, ed è importante saperlo: `index.js:157` passa `workspaceDir`: `api.config?.agents?.defaults?.workspace ?? process.cwd()`. La `tmpdir` (`withTempWorkspace`, prefix `openclaw-llm-task-`) serve **solo** per `session.json`. Il run gira nel workspace reale dell'agente.

2.2 Quali harness lo supportano, e cosa succede quando no

Il gate è esplicito e **lancia**, non degrada:

```
// dist/prepare.runtime-CM3Uj6Uh.js:320
if (params.disableTools === true && backendResolved.nativeToolMode === "always-on" &&
    !sideQuestionDisablesNativeTools)
  throw new Error(`CLI backend ${backendResolved.id} cannot run with tools disabled because it exposes
    native tools`);
```

I due backend che dichiarano `nativeToolMode: "always-on"` in tutta la dist:

- `dist/cli-backend-C4iY7FFY.js:21` — **Claude CLI**
- `dist/cli-backend-Dym4FoF1.js:48` — **google-gemini-cli**

Con questi due, `llm-task` fallisce con errore esplicito. Gli altri backend CLI non dichiarano il campo e passano. Per i provider API nativi il `disableTools` è onorato a monte (`dist/attempt-tool-construction-plan-B_ioY5W5.js:126-131`, `dist/selection-JInn13lc.js:5265-5268`).

2.3 Validazione dello schema: `TypeBox`, un colpo solo, zero retry

Libreria: **`TypeBox`**, non `Ajv` (`dist/schema-validator-BRkrm3P2.js:5-6`, Compile da `typebox/compile`). `llm-task` chiama con `cache: false` (`index.js:186`) e senza `applyDefaults`: i default dello schema non vengono applicati.

Fuori schema -> errore secco:

```
// dist/extensions/llm-task/index.js:188-191
throw new Error(`LLM JSON did not match schema: <errori>`);
```

`grep -n "retry\|attempt" dist/extensions/llm-task/index.js -> zero occorrenze. Un solo tentativo. Nessun repair-loop, nessun re-prompt.`

Lo stripping delle code fence è minimo e ancorato:

```
// dist/extensions/llm-task/index.js:21-26
const m = trimmed.match(/^````(?:json)?\s*([\s\S]*?)\s*```$/i);
```

Toglie una fence **solo se avvolge l'intera risposta**. Un preambolo tipo «Ecco il JSON:» manda tutto in `JSON.parse` e produce `"LLM returned invalid JSON"`.

2.4 La temperature è onorata? Dipende, e in due casi sparisce in silenzio

Il doc è onesto (`docs/tools/llm-task.md:79`): «Best-effort; not all providers honor it». Nel codice:

Transport	Riga	Comportamento
OpenAI chat/completions	<code>dist/openai-transport-stream-B0WkSqXp.js:3431</code>	passthrough, nessun guard
OpenAI Responses	<code>:2146</code>	passthrough
Google	<code>dist/transport-stream-D1FIItGE4.js:382</code>	passthrough
Ollama	<code>dist/stream-DwcvMYU5.js:708</code>	passthrough
Anthropic Messages	<code>dist/provider-stream-CqSx37Uy.js:675</code>	scartata se <code>thinkingEnabled</code>
Codex Responses nativo	<code>:2065-2072, :2083</code>	cancellata , insieme a <code>max_output_tokens</code> , <code>top_p</code> , <code>service_tier</code>

2.5 Il timeout è mascherato da errore di formato

Default 30.000 ms. Alla scadenza `runEmbeddedAgent` **ritorna, non lancia**:

```
// dist/embedded-agent-DGUuxGR2.js:4229-4234
return { payloads: [... , { text: timeoutText, isError: true }], ... }
```

Ma `collectText` scarta i payload con `isError` (`index.js:28`). Quindi:

- timeout senza output parziale -> testo vuoto -> `"LLM returned empty output"` (`index.js:172`)
- timeout con output parziale -> il testo parziale passa il filtro -> `"LLM returned invalid JSON"`

Il chiamante non vede mai la parola "timeout". Il `meta` che conterrebbe la causa non viene letto: `index.js:171` legge solo `result.payloads`. Per un deployment aziendale questo va compensato registrando la durata a monte della chiamata.

2.6 Blocco di config

```
{
  "plugins": {
    "entries": {
      "llm-task": {
        "enabled": true,
        "config": {
          "defaultProvider": "deepseek",
          "defaultModel": "deepseek-v4-flash",
          "allowedModels": ["deepseek/deepseek-v4-flash"],
          "maxTokens": 800,
          "timeoutMs": 30000
        }
      }
    }
  },
  "tools": { "alsoAllow": ["llm-task"] }
}
```

`allowedModels` è l'unico vincolo forte del plugin: una richiesta per un modello fuori lista viene rifiutata (`index.js:110`).

3. Lobster: pipeline in una chiamata, e i suoi limiti veri

3.1 Non è installato qui

```
$ ls /Users/apple/.local/lib/node_modules/openclaw/dist/extensions | grep -i "lobster\|llm"
litellm
llm-task
vllm

$ openclaw plugins list 2>&1 | grep -ci "lobster"
0
```

È un plugin esterno del catalogo ufficiale: `"name": "@openclaw/lobster", "minHostVersion": ">=2026.4.25"` (`dist/official-external-plugin-catalog-ph3rbXr3.js:1114-1128`).

Le righe di codice citate sotto vengono dal tarball `npm @openclaw/lobster@2026.7.1` scaricato in sola lettura in `/tmp/lob` (`npm pack`, nessuna installazione). **Non sono file presenti su questa macchina.**

3.2 Cosa promette, e come si abilita

Il doc è chiaro (`docs/tools/lobster.md:21-26`): una chiamata invece di molte, approvazioni incorporate, ripresa con token. Runner **in-process**, nessun sottoprocesso `lobster` (:71-75). Disabilitato nei contesti sandboxed (:113).

```
{ "tools": { "alsoAllow": ["lobster"] } }
```

Parametri (:276-282): `timeoutMs` default `20000`, `maxStdoutBytes` default `512000`, `cwd` relativo obbligatorio.

3.3 Come si innesta `llm-task` — e il limite documentato è diverso da quello atteso

La richiesta ipotizzava che la garanzia si rompa quando uno stage passa i dati a un processo esterno via `exec`. **Il limite dichiarato nel doc è un altro**, e va riportato per primo (`docs/tools/lobster.md:179-193`):

*«The bundled Lobster plugin runs workflows **in-process** inside the gateway. In that embedded mode, `openclaw.invoke` does **not** automatically inherit a gateway URL/auth context for nested OpenClaw CLI tool calls. That means this pattern is **not currently reliable in the embedded runner**: `openclaw.invoke --tool llm-task --action json --args-json '{ ... }'`»*

Il doc raccomanda quindi (:212-216) o una chiamata diretta a `llm-task` **fuori** da Lobster, o stage che non usano `openclaw.invoke`.

L'esempio ufficiale, valido **solo** per la CLI standalone (`docs/tools/lobster.md:195-210`):

```
openclaw.invoke --tool llm-task --action json --args-json '{
  "prompt": "Given the input email, return intent and draft.",
  "thinking": "low",
  "input": { "subject": "Hello", "body": "Can you help?" },
  "schema": {
    "type": "object",
    "properties": { "intent": { "type": "string" }, "draft": { "type": "string" } },
    "required": ["intent", "draft"],
    "additionalProperties": false
  }
}'
```

3.4 L'ipotesi sull'exec è corretta, e la prova sta nel codice

Ogni step Lobster è un comando esterno. Come viene eseguito:

```
// /tmp/lob/package/node_modules/@clawdbot/lobster/dist/src/shell.js:18-21
// POSIX: /bin/sh con ["-lc", command]
```

`-l` è **login shell**: carica i profili utente, e la stringa passa per l'espansione completa della shell. La shell si usa con `--shell`, **oppure automaticamente** se c'è un solo argomento contenente uno spazio (`exec.js:35`).

Allowlist di comandi: non esiste. Grep di `allowlist|allowList|denylist` su tutto `@clawdbot/lobster/dist/src`: zero occorrenze. L'unico tracciamento è dichiarativo (`exec.js:16`, `sideEffects: ["local_exec"]`) e il solo consumatore è una stampa in `commands_list.js:38`.

`maxStdoutBytes` **non copre l'output del processo esterno**. I sink limitati (`index.js:150-158`) sono agganciati a `ctx.stdout/ctx.stderr` del contesto pipeline (:228-229). `exec.js:87-95` accumula lo stdout del figlio in una stringa JS senza tetto (`stdout += d`) e lo restituisce come item della pipeline. Un comando che sputa gigabyte gonfia la memoria del Gateway senza incontrare il cap.

Cosa è **effettivamente imposto**: timeout con `controller.abort()` propagato fino a `spawn` (il figlio viene davvero ucciso), jail sul cwd (relativo, `path.relative` non deve iniziare con `..`), disattivazione in sessione sandboxed (`index.js:672`, `if (ctx.sandboxed) return null;`), rifiuto degli step `needs_input`.

Sintesi onesta: uno stage che chiama un processo esterno è controllato **solo** su timeout, cwd e sandbox-flag. Su *cosa* esegue e *quanto* produce è una scatola nera.

3.5 Il resume token non è firmato e non scade

```
// /tmp/lob/package/node_modules/@clawdbot/lobster/dist/src/token.js:1-13
export function encodeToken(obj) {
  const json = JSON.stringify(obj);
  return Buffer.from(json, "utf8").toString("base64url");
}
```

Nessun HMAC, nessun `iat/exp`. Contenuto: `{ protocolVersion: 1, v: 1, kind, stateKey }`. La validazione (`resume.js:146-160`) controlla solo i due numeri di versione. **Chi conosce lo stateKey può fabbricare un token valido**. Persistenza in `~/lobster/state` (o `LOBSTER_STATE_DIR`), file o600, scritture atomiche. Scadenza: nessuna — grep di `ttl|expire|maxAge` trova solo la *stringa* del messaggio «Approval ID ... not found or expired».

3.6 Quando conviene

Conviene quando la pipeline è **fatta di comandi che controlli tu**, e il valore che cerchi è il cancello di approvazione con ripresa. Non conviene come garanzia di determinismo: il determinismo lo dà il codice dei tuoi comandi, non Lobster. E in un deployment aziendale il resume token va trattato come una credenziale: chi legge la directory di stato può riprendere il flusso.

4. Il cancello anti-invenzione

Questo è il punto centrale, ed è anche quello dove la premessa della richiesta va corretta.

4.1 Catalogo hook: 40 nomi, e uno solo blocca davvero l'uscita

Autorità: dist/hook-types-DQ9eTy2x.d.ts:349, costante PLUGIN_HOOK_NAMES. Il doc (docs/plugins/hooks.md:91-166) li raggruppa e marca in **grassetto** quelli che accettano una decisione. I quattro che interessano:

Hook	Può decidere?	Ritorno
after_tool_call	NO — osservazione pura	Promise<void> void (dist/hook-types-DQ9eTy2x.d.ts:1112)
before_agent_reply	short-circuit	{handled, reply?, reason?}
before_agent_finalize	solo revise/finalize	{action?: "continue" "revise" "finalize", reason?, retry?}
message_sending	cancel — terminale	{content?, cancel?, cancelReason?, metadata?}

4.2 before_agent_finalize è fail-open in sei modi. Nessuno è un bug: è il progetto.

(1) Un'eccezione nell'handler lascia passare.

```
// dist/lifecycle-hook-helpers-BwL6869q.js:120-123
} catch (error) {
  log.warn(`before_agent_finalize hook failed: ${String(error)}`);
  return { action: "continue" };
}
```

(2) Il timeout lascia passare. Budget default 15 secondi:

```
// dist/hook-runner-global-BmIrGllG.js:254-259
const DEFAULT_MODIFYING_HOOK_TIMEOUT_MS_BY_HOOK = {
  before_agent_run: 15e3,
  before_agent_start: 15e3,
  before_agent_finalize: 15e3,
  ...
};
```

Il doc lo conferma (docs/plugins/hooks.md:431-432): «Handlers have a 15s default budget; on timeout, OpenClaw logs the failure and continues with the original final answer.»

(3) C'è un tetto host cablato a 3, e all'ultimo giro l'hook non viene nemmeno chiamato.

```
// dist/embedded-agent-DGUuxGR2.js:1774
const MAX_BEFORE_AGENT_FINALIZE_REVISIONS = 3;

// dist/selection-JInn13lc.js:13332-13336
const maxRevisionAttempts = params.maxBeforeAgentFinalizeRevisions ?? 0;
if (maxRevisionAttempts > 0 && (params.beforeAgentFinalizeRevisionAttempts ?? 0) >= maxRevisionAttempts) {
  log$2.warn(`before_agent_finalize revision limit reached; finalizing ...`);
  return;
}
```

Il return precede la chiamata a runAgentHarnessBeforeAgentFinalizeHook (:13338). Non esiste chiave di configurazione per alzarlo: è una costante.

(4) Il retry.maxAttempts del plugin ha default 1.

```
// dist/lifecycle-hook-helpers-BwL6869q.js:141-152
const maxAttempts = typeof retry.maxAttempts === "number" && Number.isFinite(retry.maxAttempts)
  ? Math.max(1, Math.floor(retry.maxAttempts)) : 1;
...
if (nextCount > maxAttempts) continue;
```

Se nessun candidato passa -> return { action: "continue" } (:157). Il budget è per runId + idempotencyKey.

(5) Se ci sono già stati effetti collaterali, la revisione viene ignorata.

```
// dist/selection-JInn13lc.js:13371-13374
if (event.hadDeterministicSideEffect) {
  log$2.warn('before_agent_finalize requested revision after potential side effects; finalizing ...');
  return;
}
```

Il flag è alzato quando parte un'operazione asincrona (`dist/selection-JInn13lc.js:2007`) o quando c'è evidenza di consegna di un tool di messaggistica, di uno spawn di sessione accettato, o di un cron aggiunto (`:4993`). In pratica: **se l'agente ha già mandato un messaggio o creato qualcosa, il cancello si disarma da solo.**

(6) Un `finalize` di un altro plugin vince ed è sticky.

```
// dist/hook-runner-global-BmIrGllG.js:370-374
if (acc?.action === "finalize") return acc;
if (next.action === "finalize") return { action: "finalize", ... };
```

Qualunque plugin installato può neutralizzare il tuo cancello.

Più le precondizioni d'ingresso (`dist/selection-JInn13lc.js:13317, :13323`): `!willRetry || isError || incompleteTerminalAssistant || !hasAssistantVisibleText, né se aborted || promptError || timedOut || hasCompletedClientToolCall || yieldDetected || silentFinalReply.`

E c'è una settimana via, esterna al codice: issue aperta **#125802** — «successful provider streams can skip response hooks» (18 ago 2026, <<https://github.com/openclaw/openclaw/issues/125802>>).

Conclusione senza sconti: `before_agent_finalize` **non è un cancello. È una richiesta di revisione, best-effort.**

Il testo che viene iniettato nella passata extra, per completezza (`dist/embedded-agent-DGUuxGR2.js:1773`):

```
"Before accepting the previous final answer, apply this revision request and produce the revised final answer. Do not repeat completed work or rerun tools unless the request explicitly requires it."
```

4.3 `message_sending` è l'unico blocco duro

```
// dist/dispatch-DnzGTpPs.js:2521
if (result?.cancel) return null;

// dist/deliver-BdKtkX_b.js:1350-1360
if (hookResult.cancelled) {
  recordPayloadOutcome(suppressedPayloadOutcome({ ..., reason: "cancelled_by_message_sending_hook" }));
  continue;
}
```

Semantica terminale (`dist/hook-runner-global-BmIrGllG.js:773-787`): `shouldStop: (result) => result.cancel === true`, con `stickyTrue`. Il doc (`docs/plugins/hooks.md:534-535`): «`message_sending` with `cancel: true` is terminal. `message_sending` with `cancel: false` is treated as no decision.»

Due limiti da progettare intorno:

1. Nel percorso `dispatch-DnzGTpPs.js:2519-2522` all'hook arrivano **solo** `{content, to}`, e il ctx **non porta** `runId` — a differenza di `reply_payload_sending` che lo inietta esplicitamente (`:2540`). La correlazione con le prove va fatta su `sessionKey`, oppure si sposta il cancel su `reply_payload_sending`.
2. `dispatch-DnzGTpPs.js:2520`: `if (!payload.text) return payload;` — **un payload senza testo (solo media) bypassa l'hook.**

4.4 `before_agent_reply`: funziona, ma non serve a questo

Può davvero produrre una risposta senza chiamare il modello:

```
// dist/get-reply-OTG64ybi.js:5622-5625
if (hookResult?.handled) {
  logResolverTiming("completed", "before_agent_reply_hook");
  return hookResult.reply ?? { text: "NO_REPLY" };
}
```

Il `return` precede `runPreparedReply (:5637)`. Ma l'evento porta solo `{cleanedBody: string}` (`dist/hook-types-DQ9eTy2x.d.ts:416-418`) e gira **prima** del turno, quando nessuna prova esiste. Utile per risposte deterministiche a comandi noti; inutile come cancello.

4.5 `allowConversationAccess`: il percorso della richiesta è sbagliato

Non esiste `hooks.allowConversationAccess`. La sezione `hooks.*` di primo livello è tutt'altro: è l'ingresso webhook HTTP (`hooks.enabled`, `hooks.path`, `hooks.token`, `hooks.mappings []`, `hooks.gmail.*`, `hooks.internal.*`).

Il percorso reale è **per plugin**:

```
plugins.entries.<plugin-id>.hooks.allowConversationAccess
```

Fonti: `dist/schema-DRY01XBt.js:819`, `dist/zod-schema-09ml_nmo.js:792` (`boolean().optional()`, nessun default), `docs/plugins/hooks.md:452-468`.

Il gate, con i due default asimmetrici:

```
// dist/registry-B8eQDFB4.js:4225-4244
if (isConversationHookName(effectiveHookName)) {
  const explicitConversationAccess = policy?.allowConversationAccess;
  if (record.origin !== "bundled" && explicitConversationAccess !== true) { /* blocked */ return; }
  if (record.origin === "bundled" && explicitConversationAccess === false) { /* blocked */ return; }
}
```

- Plugin **non-bundled** (il caso di un plugin aziendale): default **BLOCCATO**, serve `true` esplicito.
- Plugin **bundled**: default consentito, `false` lo blocca.
- Il blocco è **silenzioso a runtime**: l'hook non viene registrato, resta solo una diagnostica `warn`.

I sette hook coperti (`dist/hook-types-DQ9eTy2x.d.ts:386`, `CONVERSATION_HOOK_NAMES`): `before_model_resolve`, `before_agent_reply`, `llm_input`, `llm_output`, `before_agent_finalize`, `agent_end`, `before_agent_run`.

`message_sending`, `after_tool_call` e `before_tool_call` **NON sono nella lista** e non richiedono il flag. Solo il ramo "revise" del cancello lo richiede.

Prova grezza — `openclaw config get hooks` sull'installazione reale, che conferma che quel namespace è un'altra cosa:

```
{
  "internal": {
    "enabled": true,
    "entries": {
      "boot-md": { "enabled": true },
      "bootstrap-extra-files": { "enabled": true },
      "command-logger": { "enabled": true },
      "compaction-notifier": { "enabled": true },
      "session-memory": { "enabled": true }
    }
  }
}
```

4.6 Come accede l'hook alle prove raccolte dai tool

Non esiste un registro trajectory interrogabile da un hook. La trajectory è un bundle di export su disco, non un'API runtime. Le tre fonti reali:

1. `after_tool_call` — payload per chiamata (`dist/hook-types-DQ9eTy2x.d.ts:738-746`): `{toolName, params, runId?, toolCallId?, result?, error?, durationMs?}`. Dispatch in `dist/hook-helpers-GgBV1rQp.js:12-39`, best-effort, fallimenti solo loggati. **Ritorno void: qui si registra, non si blocca.**
2. `before_agent_finalize.messages` — il transcript del run, costruito a `dist/selection-JInn13lc.js:13324`. È **l'unico punto in cui l'hook vede insieme** la risposta finale (`lastAssistantMessage`) e la catena di prove. Tipizzato `unknown[]`: il parsing è a carico del plugin.
3. **Stato di sessione plugin-owned** — `api.session.state.registerSessionExtension(...)` + `ctx.getSessionExtension(namespace)` (`docs/plugins/hooks.md:473-490`). È il canale corretto per portare l'accumulo da `after_tool_call` fino a `message_sending`.

Trappola: i `details` dei tool result non sono prova affidabile. OpenClaw li strippa prima del replay e li tronca in persistenza con `persistedDetailsTruncated: true` (docs/plugins/hooks.md:302–310). Le prove vanno lette da `content`.

4.7 L'architettura del cancello: tre stadi, non uno

```
after_tool_call      -> registra le prove (nessun flag richiesto, non blocca)
    | stato di sessione, chiavato su ctx.runId + ctx.sessionKey
before_agent_finalize -> confronta risposta vs prove, chiede revisione
    | richiede allowConversationAccess:true – CORRETTIVO, non bloccante, tetto 3
message_sending      -> {cancel:true} oppure riscrittura fail-closed
    | TERMINALE – è l'unico punto che ferma davvero l'uscita
```

Config minima:

```
{
  "plugins": {
    "entries": {
      "cancello-prove": {
        "enabled": true,
        "hooks": {
          "allowConversationAccess": true,
          "timeouts": { "before_agent_finalize": 30000 }
        }
      }
    }
  }
}
```

`plugins.entries.<id>.hooks.timeouts.<hookName>` sovrascrive `hooks.timeoutMs`, che sovrascrive il `timeoutMs` scritto nel plugin. Valore massimo 600.000 ms (docs/plugins/hooks.md:82–85).

Scheletro del plugin — struttura copiata da docs/plugins/hooks.md:22–49, con i tre hook al posto giusto:

```
import { definePluginEntry } from "openclaw/plugin-sdk/plugin-entry";

export default definePluginEntry({
  id: "cancello-prove",
  name: "Cancello prove",
  register(api) {
    // 1. registra – non blocca, nessun flag richiesto
    api.on("after_tool_call", async (event, ctx) => {
      // accumula event.toolName / event.params / event.result
      // in api.session.state, chiavato su ctx.runId
    }, { priority: 50 });

    // 2. chiede revisione – richiede allowConversationAccess:true
    api.on("before_agent_finalize", async (event, ctx) => {
      // confronta event.lastAssistantMessage con le prove accumulate
      return {
        action: "revise",
        reason: "Affermazioni non supportate dai dati raccolti.",
        retry: { instruction: "...", idempotencyKey: `${ctx.runId}:grounding`, maxAttempts: 2 },
      };
    }, { priority: 50 });

    // 3. blocca – terminale. Priorità negativa perché nessuno lo scavalchi.
    api.on("message_sending", async (event, ctx) => {
      // se ancora non supportato: sostituire il testo, oppure
      return { cancel: true, cancelReason: "grounding_failed" };
    }, { priority: -100 });
  },
});
```

Il manifest `openclaw.plugin.json` non ha un campo `contracts.hooks`: gli hook tipizzati non sono manifest-gated (verificato su docs/plugins/manifest.md). `defineToolPlugin` **non** serve, e anzi non basta: docs/plugins/tool-plugins.md:11–13 dichiara «no channel, model provider, **hook**, service, or setup backend».

4.8 Esiste già: `llm-grounded` su ClawHub — conviene adottarlo

Verificato dal vivo, output letterale di `openclaw plugins search llm-grounded`:

ClawHub plugins (1)
llm-grounded code-plugin | community | v0.2.4 – Deterministic grounding and voice contract for LLM agents.
Framework-independent core, with an OpenClaw plugin adapter included.
Install: `openclaw plugins install clawhub:llm-grounded`

- ClawHub: <<https://clawhub.ai/polley-mister/plugins/llm-grounded>> – v0.2.4
- Repo: <<https://github.com/polley-mister/llm-grounded>> – ultimo commit **7 ago 2026**, `package.json` a **0.3.5** (la scheda ClawHub è indietro)
- **Non è su npm:** `https://registry.npmjs.org/llm-grounded -> {"error": "Not found"}`. È un pacchetto ClawHub-nativo.

Registra otto hook, nell'ordine asserito dal suo stesso `tests/manifest.test.mjs`:

```
before_prompt_build, before_tool_call, after_tool_call, before_agent_finalize,  
before_message_write, agent_end, reply_payload_sending, message_sending
```

Fa esattamente l'architettura ricavata sopra, e fail-closed: `before_agent_finalize` restituisce `{action:"revise", retry:{maxAttempts:1}}` quando un turno che doveva recuperare dati non l'ha fatto; dopo la revisione limitata, `reply_payload_sending` e `message_sending` **riscrivono il testo in uscita** con una frase fail-closed e rimuovono `mediaUrls/presentation`. Il test asserisce che entrambi i `_sending` si registrino con `priority < 0` «so it cannot be bypassed».

Riserva da dichiarare al cliente: il repo GitHub pubblica solo `types/`, `tests/`, `doc` e `manifest`. La cartella `src/` è dichiarata in `package.json.files` ma **non è committata su GitHub**. La lista degli hook viene dal test che asserisce la registrazione, non dalla lettura di `register()`. Per un uso aziendale va scaricato il pacchetto ClawHub e **letto il sorgente reale** prima dell'adozione.

Alternativa pubblicata su npm: `@vainplex/openclaw-governance v0.11.2`

(<<https://www.npmjs.com/package/@vainplex/openclaw-governance>>, 166 download/mese, ultimo push 14 ago 2026). Suite di governance completa; il suo Response Gate ispeziona il `toolCallLog` associato al messaggio e può restituire `{cancel:true}` su `message_sending`. Non usa `before_agent_finalize`.

Cosa NON usare, pur avendo nomi promettenti (verificato leggendo i `api.on(...)` reali): `vericlaw` (un solo hook, `before_prompt_build`, è un router verso un'app macOS); `openclaw-accountability` (solo `after_tool_call`, verifica artefatti su disco, mai la prosa); `proofrail-openclaw` (9 hook, ma **nessun** `before_agent_finalize` né `*_sending`: gatea i tool, non la risposta); `@traceproof/openclaw-traceproof-runtime` (attesta *che* un messaggio è stato mandato, dopo l'invio); `critiqor-openclaw` (dichiaratamente passivo); `@laninthesdran/openclaw-reliability-engine` (**zero hook**, solo tool che l'agente può scegliere di non chiamare).

Raccomandazione: partire da `llm-grounded`, dopo audit del sorgente. Riscriverlo da zero significa reimplementare sei mitigazioni di fail-open che qualcuno ha già trovato.

5. Latenza: dove si perdono i secondi su una domanda banale

5.1 Prima di tutto, la misura

Il peso del prompt di sistema non è un'opinione: OpenClaw lo calcola e lo persiste come `systemPromptReport` (costruito in `dist/prepare.runtime-CM3Uj6Uh.js:788`, persistito in `dist/session-store-QMAG1JNr.js:113`). Estratto letterale da una sessione reale di questa installazione (`opencode-go/deepseek-v4-flash`):

```
"systemPrompt": {
  "chars": 58315,
  "projectContextChars": 31084,
  "nonProjectContextChars": 27231
},
"bootstrapMaxChars": 20000,
"bootstrapTotalMaxChars": 60000,
"injectedWorkspaceFiles": [
  { "name": "AGENTS.md", "rawChars": 6368 },
  { "name": "SOUL.md", "rawChars": 2123 },
  { "name": "TOOLS.md", "rawChars": 6529 },
  { "name": "IDENTITY.md", "rawChars": 181 },
  { "name": "USER.md", "rawChars": 211 },
  { "name": "MEMORY.md", "rawChars": 15097 }
],
"skills": { "promptChars": 12437, "entries": [ ...36 voci... ] },
"tools": { "listChars": 0, "schemaChars": 31833, "entries": [ ...47 voci... ] },
"currentTurn": { "promptChars": 13, "runtimeContextChars": 0 }
```

Circa 90.148 caratteri fissi per turno — 58.315 di system prompt più 31.833 di schemi tool — per un prompt utente da 13 caratteri. Le voci più grasse:

- `MEMORY.md`: 15.097 caratteri, il 48,6% del project context
- 36 skill: 12.437 caratteri, il 45,7% del blocco non-project — e sono solo metadati
- tool `message`: 6.061 caratteri di schema, 109 proprietà; `browser`: 3.624

5.2 Il thinking: un solo interruttore reale su DeepSeek V4

```
// dist/provider-stream-shared-B4Hm1tKd.js:1477-1479
function resolveDeepSeekV4ReasoningEffort(thinkingLevel) {
  return thinkingLevel === "xhigh" || thinkingLevel === "max" ? "max" : "high";
}
```

`minimal`, `low`, `medium`, `high` producono **lo stesso identico** `reasoning_effort: "high"`. Abbassare da `medium` a `low` non cambia un byte. L'unica leva è `off`, che manda `thinking: {type: "disabled"}` e rimuove `reasoning_content` dalla storia replicata (:1505-1511).

Trappola sulla risoluzione (`docs/tools/thinking.md:43-49`): l'**override di sessione** sta al livello 2 e batte la config al livello 3. `thinkingDefault: "off"` in `openclaw.json` non basta se qualcuno ha fatto `/think medium: serve /think default`.

Seconda trappola: togliere `thinkingDefault` non porta a `off`. Il plugin DeepSeek dichiara `defaultLevel: "high"` per i V4.

5.3 Checklist, ordinata per impatto, con le chiavi verificate

#	Azione	Chiave (verificata nello schema)
1	Spegnere il thinking	<code>agents.defaults.thinkingDefault: "off"</code> — enum <code>off minimal low medium high xhigh adaptive max ultra</code> ; per agente <code>agents.list[].thinkingDefault</code>
2	Sfoltire le skill	nessuna chiave: si agisce sui file/root delle skill. 12.437 caratteri/turno misurati
3	Sfoltire i tool	<code>agents.list[].tools.allow / .deny</code> , <code>tools.deny</code>
4	Ridurre il project context	<code>agents.defaults.bootstrapMaxChars (20000)</code> , <code>agents.defaults.bootstrapTotalMaxChars (60000)</code>
5	Non far bloccare il primo turno da MCP lenti	<code>mcp.servers.<n>.enabled</code> , <code>.connectionTimeoutMs</code> , <code>.toolFilter</code> — non impostare <code>requestTimeoutMs</code> senza motivo
6	Cache del prefisso	<code>agents.defaults.params.cacheRetention</code> (dentro <code>params</code> , free-form, non validato); su proxy serve <code>models.providers.<id>.models[].compat.supportsPromptCacheKey: true</code>
7	Non ricacheare storia vecchia	<code>agents.defaults.contextPruning.mode: "cache-ttl" + .ttl</code>

#	Azione	Chiave (verificata nello schema)
8	Saltare la reiniezione su continuation	<code>agents.defaults.contextInjection: "continuation-skip"</code>
9	Tool Search — solo se il catalogo è grande	<code>tools.toolSearch.{enabled,mode,codeTimeoutMs,searchDefaultLimit,maxSearchLimit}, mode ☐ code tools directory</code> . Solo globale , non per agente
10	Coda peggiore	<code>models.providers.<id>.timeoutSeconds</code>

Chiavi che NON esistono e vanno evitate: `promptCaching`, `trajectory`, `telemetry`, `prometheus`, `llmTask`, `lobster`, `permissionMode`, `cacheRetention` come chiave tipizzata, `compat.supportsLongCacheRetention` come chiave di config.

5.4 Tre avvertenze controintuitive

Tool Search può peggiorare le cose. Sposta il costo dal pre-token ai round-trip (`search` -> `describe` -> `call`). Il doc lo dice (`docs/tools/tool-search.md:81-82, :106-108`): «Prefer direct tool exposure for small OpenClaw tool catalogs». Con 47 tool è probabilmente una perdita. E se il runtime del child Node isolato non parte, `code` degrada silenziosamente a `tools` (:74-77).

Code mode fallisce chiuso, ed è un pregio. «if code mode is enabled but the QuickJS-WASI runtime is unavailable, the run fails instead of silently falling back to broad direct tool exposure» (`docs/reference/code-mode.md:63-65`). Default `enabled: false`, runtime solo `quickjs-wasi`, mode solo `only`.

Il prompt caching probabilmente non è attivo dove pensi. Su un endpoint OpenAI-compatibile che non è `api.openai.com`, `prompt_cache_key` non viene mai auto-attivato: «this is never auto-detected for a proxy» (`docs/reference/prompt-caching.md:87`). Le voci `deepseek` nel catalogo locale non dichiarano `supportsPromptCacheKey`. Discrepanza da segnalare: `docs/reference/prompt-caching.md:88` consiglia `compat.supportsLongCacheRetention`, ma quella chiave **non è nello schema di config** — arriva solo dal catalogo del provider.

6. Determinismo del modello

6.1 La `temperature` non ha una chiave: viaggia in un contenitore free-form

`grep -o '"temperature"' /tmp/oc-schema.json -> 0 occorrenze`. La `temperature` non è dichiarata da nessuna parte nello schema. Passa perché `params` accetta qualunque cosa:

```
{"type":"object","propertyNames":{"type":"string"},"additionalProperties":{}}
```

Stessa forma per `agents.defaults.params`, `agents.list[].params`, `agents.defaults.models.<ref>.params`, `models.providers.<id>.params`, `models.providers.<id>.models[].params`.

Ordine di fusione (l'ultimo vince), `dist/extra-params-cce1g0up.js:87-142`:

```
agents.defaults.params -> agents.defaults.models["provider/model"].params -> agents.list[].params
```

Estrazione, `dist/extra-params-cce1g0up.js:310-311`:

```
if (typeof extraParams.temperature === "number") streamParams.temperature = extraParams.temperature;
if (typeof extraParams.topP === "number") streamParams.topP = extraParams.topP;
```

L'unico filtro è `typeof === "number"`. `"0.2"` come stringa viene **ignorato in silenzio**. E non c'è controllo di range: la validazione $0 \leq \text{temperature} \leq 2$ esiste (`dist/openai-tool-choice-CE6vvN3p.js:127-133`) ma è chiamata **solo** da `dist/openai-http-9urlXl0E.js:530`, cioè il server OpenAI-compatibile che *OpenClaw espone in ingresso*. Non sta sul percorso in uscita. Un `temperature: 47` in `openclaw.json` non lo ferma nessuno.

Blocco di config corretto:

```
{
  "agents": {
    "defaults": {
      "models": {
        "deepseek/deepseek-v4-flash": {
          "params": { "temperature": 0.13, "topP": 1 }
        }
      },
      "thinkingDefault": "off"
    }
  }
}
```

6.2 compat.supportsTemperature è un flag morto

La chiave esiste nello schema (`models.providers.<id>.models[].compat.supportsTemperature`), sopravvive alla validazione zod, e **non ha alcun effetto**. Tutte e sei le occorrenze nel pacchetto sono dichiarazioni di tipo o di schema:

```
dist/zod-schema-core-DviqqtPj.js:118:    supportsTemperature: boolean().optional(),
dist/types-CFIUY_La.d.ts:370:    supportsTemperature?: boolean;
dist/types.models-BvJnk7Su.d.ts:10
dist/sessions-D8qGY7uC.js:1209
dist/plugin-sdk/config-schema.d.ts:609
node_modules/@openclaw/ai/dist/types-DRgdPqaZ.d.mts:434
```

Zero siti di lettura. Per contrasto, nello stesso file dove `supportsPromptCacheKey` viene usato, la temperature non ha nessun guard:

```
// dist/openai-transport-stream-B0WkSqXp.js:3427-3431
if (compat.supportsPromptCacheKey && promptCacheKey) {           // <- letto
  params.prompt_cache_key = promptCacheKey;
  ...
}
if (options?.temperature !== void 0) params.temperature = options.temperature; // <- nessun gate
```

Chi lo imposta credendo di proteggere un modello che rifiuta `temperature` prende comunque un 400 dal provider.

I due filtri per modello che invece esistono davvero non si chiamano così:

- **Codex Responses nativo** — `OPENAI_CODEX_RESPONSES_UNSUPPORTED_PARAMS` include `temperature`, `top_p`, `max_output_tokens`, `metadata`, `prompt_cache_retention`, `service_tier`, tutti cancellati con `delete` (`dist/openai-transport-stream-B0WkSqXp.js:2065-2072, :2083`)
- **Moonshot** — `MOONSHOT_FIXED_SAMPLING_FIELDS` (`dist/moonshot-thinking-y4RaKUsA.js:6-12`)
- **Anthropic con thinking acceso** — `dist/provider-stream-CqSx37Uy.js:675`: la temperature è scartata se `options.thinkingEnabled`

6.3 --raw-stream esiste, ma non prova quello che serve

Output letterale di `openclaw gateway --help`, estratto:

```
--raw-stream          Log raw model stream events to jsonl (default:
                      false)
--raw-stream-path <path> Raw stream jsonl path
```

Entrambi i flag esistono. Diventano variabili d'ambiente (`dist/run-CSvF3r1e.js:1161-1163`):

```
if (opts.rawStream) process.env.OPENCLAW_RAW_STREAM = "1";
const rawStreamPath = toOptionString(opts.rawStreamPath);
if (rawStreamPath) process.env.OPENCLAW_RAW_STREAM_PATH = rawStreamPath;
```

Ma l'unico scrittore è `appendRawStream` (`dist/selection-JInn13lc.js:2816`), **con 5 call site** (`:3337, :3368, :3388, :3575, :3601`), **e ognuno emette solo eventi** `assistant_text_stream` / `assistant_thinking_stream`. È puro lato risposta. Il corpo della richiesta non ci passa mai. **Non si può provare la temperature con `--raw-stream`.**

6.4 La procedura che invece funziona

Lo strumento giusto è il debug proxy, che dichiara di catturare proprio il seam di uscita. Output letterale di `openclaw proxy coverage`:

```
{
  "id": "provider-transport-fetch",
  "label": "Provider HTTP transport",
  "modulePath": "src/agents/provider-transport-fetch.ts",
  "protocols": ["http", "https", "sse"],
  "status": "captured",
  "notes": "Central provider fetch seam routes through explicit proxy overrides and records request/
    response payloads."
}
```

Procedura in sei passi:

1. Scrivere un valore **riconoscibile a occhio** (non 0 né 1) nel punto più specifico della catena:
`agents.defaults.models["<provider>/<modello>"].params.temperature = 0.13`
2. Riavviare il Gateway. Un solo restart.
3. `openclaw proxy purge` per partire da un catturato vuoto.
4. `openclaw proxy start --port <p>`
5. Un turno breve sul modello bersaglio.
6. `openclaw proxy sessions ->` prendere l'id; `openclaw proxy blob <id> ->` cercare `"temperature":0.13` nel body.

Controprova a costo zero, senza rete: il lato config è già loggato. `dist/extra-params-cce1g0up.js:337-343` emette a livello debug:

```
creating streamFn wrapper with params: {"temperature":0.13}
```

I due insieme chiudono la catena: il log prova `config->streamParams`, il proxy prova `streamParams->filo`.

Verifica negativa da fare nella stessa sessione: ripetere su un modello Anthropic con thinking acceso e controllare che `temperature` sia **assente** dal body.

6.5 DeepSeek V4 con thinking

`docs/providers/deepseek.md` **non dice nulla sulla temperature:** zero occorrenze del termine. Su reasoning e thinking (:86-92, :94-110):

«Use `/think xhigh` or `/think max` with DeepSeek V4 models to request DeepSeek's maximum `reasoning_effort`; both map to `"max"`.»

«When thinking is disabled (including the UI **None** selection), OpenClaw sends `thinking: { type: "disabled" }` and strips replayed `reasoning_content` from outgoing history, keeping the session on the non-thinking DeepSeek path.»

Il doc **non** dice l'altra metà — che tutti i livelli non-off sotto `xhigh` collassano su `"high"`. Quella si legge solo nel codice (§5.2).

Catalogo locale del plugin DeepSeek, `compat` dichiarati per V4 Flash e V4 Pro:

```
"compat": {
  "supportsUsageInStreaming": true,
  "supportsReasoningEffort": true,
  "maxTokensField": "max_tokens"
}
```

Nessun `supportsTemperature`, nessun `supportsPromptCacheKey`. Profilo thinking: sette livelli (`off`, `minimal`, `low`, `medium`, `high`, `xhigh`, `max`), `defaultLevel: "high"`.

7. Isolamento multiutente

7.1 La risposta scomoda, in una citazione

Prima di ogni configurazione, va detto al cliente cosa il progetto dichiara di sé (docs/gateway/security/index.md:8-14):

«**Personal assistant trust model.** This guidance assumes one trusted operator boundary per gateway (single-user, personal-assistant model). OpenClaw is **not** a hostile multi-tenant security boundary for multiple adversarial users sharing one agent or gateway. For mixed-trust or adversarial-user operation, split trust boundaries: separate gateway + credentials, ideally separate OS users or hosts.»

E l'elenco esplicito (docs/gateway/security/index.md:19-25):

«- Supported: one user/trust boundary per gateway (prefer one OS user/host/VPS per boundary). - Not supported: one shared gateway/agent used by mutually untrusted or adversarial users. - If several untrusted users can message one tool-enabled agent, they share that agent's delegated tool authority. - Inside one Gateway, authenticated operator access is a trusted control-plane role, not a per-user tenant role. - sessionKey (session IDs, labels) is a routing selector, not an authorization token.»

Ecco perché /gateway/multi-tenant-hosting non esiste: non è un buco di documentazione, è una scelta. Per più clienti servono più Gateway, su utenti di sistema o host separati.

7.2 I tre controlli, e cosa garantisce ciascuno

Citazione diretta (docs/gateway/sandbox-vs-tool-policy-vs-elevated.md:8-12):

«1. **Sandbox** (agents.defaults.sandbox.*) decides **where tools run**. 2. **Tool policy** (tools.*, tools.sandbox.tools.*, agents.list[].tools.*) decides **which tools are available/allowed**. 3. **Elevated** (tools.elevated.*) is an **exec-only escape hatch** to run outside the sandbox when you are sandboxed.»

Le due frasi che decidono un progetto (:68-69):

«Tool policy is the hard stop: /exec cannot override a denied exec tool.» «Tool policy filters tool availability by name; it does not inspect side effects inside exec. If exec is allowed, denying write, edit, or apply_patch does not make shell commands read-only.»

7.3 Sandbox: cosa isola, e i suoi buchi dichiarati

docs/gateway/sandboxing.md:9,12:

«Sandboxing is off by default... The Gateway process always stays on the host; only tool execution moves into the sandbox when enabled.» «This is not a perfect security boundary, but it materially limits filesystem and process access when the model does something dumb.»

Dentro la sandbox: exec, read, write, edit, apply_patch, process, browser. **Fuori:** il Gateway, e qualunque tool passato per tools.elevated.

Default Docker: readOnlyRoot: true, capDrop: ["ALL"], network: "none". network: "host" è bloccato; container:<id> bloccato salvo docker.dangerouslyAllowContainerNamespaceJoin.

I tre buchi da dichiarare:

- I bind mount **bucano** la sandbox (:215): «Binds bypass the sandbox filesystem: they expose host paths with whatever mode you set». Montare /var/run/docker.sock «effectively hands host control to the sandbox».
- docker.env è leggibile con docker inspect da chiunque abbia accesso al daemon (:349).
- Le tool policy si applicano **prima**: «If a tool is denied globally or per-agent, sandboxing doesn't bring it back» (:356).

Nota sullo schema: agents.defaults.sandbox.backend è {"type":"string","minLength":1} — **nessun enum, nessuna validazione dei valori**. Il doc documenta docker, ssh, openshell, ma nel codice l'unico confronto è backend === "docker" (dist/sandbox-cli-DhN0VhUT.js:130). Un backend scritto male probabilmente non viene rifiutato dalla validazione.

7.4 toolsBySender: filtra in uscita, non ri-aggiunge

Domanda diretta, risposta dal codice. La pipeline è puramente sottrattiva:

```
// dist/tool-policy-pipeline-C3ed0W1F.js:209, 240-241, 256
let filtered = params.tools;
...
const before = filtered;
filtered = filterToolsByPolicy(before, expanded);
...
return filtered;

// dist/agent-tools.policy-YD9HuYg0.js:87-90
function filterToolsByPolicy(tools, policy) {
  if (!policy) return tools;
  return tools.filter((tool) => isToolAllowedByPolicyName(tool.name, policy));
}
```

E toolsBySender è l'ultimo step (dist/tool-policy-pipeline-C3ed0W1F.js:195-196): opera su una lista già ristretta e non può resuscitare nulla.

alsoAllow non fa eccezione: unionAllow con base vuota produce ["*", ...extra] (dist/sandbox-tool-policy-ClB7s2K0.js:9-22), cioè «lascia passare quello che i livelli precedenti avevano già ammesso, più questi nomi». Il deny continua ad applicarsi.

Un dettaglio di precedenza che sorprende: un toolsBySender per agente **sostituisce** quello globale, con return anticipato — il globale non viene nemmeno valutato (dist/sender-tool-policy-BAIGCLiF.js:22-30).

Chiavi ammesse, con ordine di match: channel:<channelId>:<senderId> -> id: -> e164: -> username: -> name: -> * (dist/group-policy-BiTzkkCF.js:128-153). Le chiavi legacy senza prefisso emettono un warning e valgono come id:.

7.5 workspaceOnly per agente può ALLENTARE il globale

Questa rompe l'aspettativa «ogni livello può solo restringere»:

```
// dist/local-roots-CAoJyC6u.js:19-22
function resolveToolFsConfig(params) {
  const globalFs = cfg?.tools?.fs;
  return { workspaceOnly: (cfg && params.agentId ? resolveAgentConfig(cfg, params.agentId)?.tools?.fs :
    void 0)?.workspaceOnly ?? globalFs?.workspaceOnly };
}
```

L'operatore è ?? : un valore per agente **esplicito** vince, **incluso** false. tools.fs.workspaceOnly: true globale non è un pavimento.

Default divergenti da non confondere: tools.exec.applyPatch.workspaceOnly default true; tools.fs.workspaceOnly default false.

7.6 Exec: le modalità, e come si spegne davvero

Le "permission mode" sono tools.exec.mode. Enum verificato nello schema:

```
{"type":"string","enum":["deny","allowlist","ask","auto","full"],"title":"Exec Mode"}
```

Semantica (docs/tools/permission-modes.md:38-44): deny blocca l'host exec; allowlist ammette solo comandi in lista e nega in silenzio i miss; ask chiede approvazione; auto passa per un classificatore e poi ricade sull'umano; full non chiede nulla.

Per spegnere exec davvero servono due chiavi, non una:

```
{
  "tools": {
    "exec": { "mode": "deny" },
    "deny": ["exec", "process", "code_execution", "group:runtime"]
  }
}
```

La prima blocca l'host exec; la seconda è l'unico *hard stop* — «/exec cannot override a denied exec tool» (docs/gateway/sandbox-vs-tool-policy-vs-elevated.md:68), e «if exec is denied by tool policy, elevated cannot override it» (docs/tools/elevated.md:102). Attenzione: deny: ["exec"] da solo non basta a rendere l'agente read-only, perché process e code_execution sono tool separati.

Approvazioni: registrate in \$OPENCLAW_STATE_DIR/exec-approvals.json (o ~/.openclaw/exec-approvals.json), pending in scadenza dopo 30 minuti. Instradamento con approvals.exec.mode (enum session|targets|both) e approvals.exec.targets[].

Caveat testuale da riportare al cliente (docs/tools/exec-approvals.md:41-45):

«Approvals reduce accidental execution risk, but are **not** a per-user auth boundary or filesystem read-only policy.»

E (exec-approvals-advanced.md:22): «Do **not** add interpreter or runtime binaries (for example python3, node, ruby, bash, sh, zsh) to safeBins.»

7.7 Threat model: i rischi residui dichiarati

Da docs/security/THREAT-MODEL-ATLAS.md, con il rischio residuo che il progetto stesso assegna:

ID	Minaccia	Residuo	Riga
T-EXFIL-003	Raccolta credenziali da skill malevola — «skills run with agent privileges»	Critical	:356-366
T-ACCESS-003	Furto di token — «tokens stored in plaintext on disk»	High	:168-178
T-IMPACT-002	Esaurimento risorse / DoS — « no per-sender rate limiting »	High	:384-394
T-DISC-002	Estrazione dati di sessione — «accessible by design»	Medium	:316-327
T-ACCESS-002	Spoofing di AllowFrom	Medium	:156-166

E la lista "non sono vulnerabilità, per progetto" (docs/gateway/security/index.md:110-121):

«- Prompt-injection-only chains without a policy, auth, or sandbox bypass. - Claims that assume hostile multi-tenant operation on one shared host or config. - "Missing per-user authorization" findings that treat sessionKey as an auth token.»

7.8 Stato reale di questa macchina

```
$ openclaw config get agents.defaults.sandbox 2>&1 | head -30
Config path not found: agents.defaults.sandbox. Run openclaw config validate to inspect config shape.
```

```
$ openclaw config get tools 2>&1 | head -12
{
  "profile": "full",
  "deny": [
    "video_generate",
    "image_generate"
  ],
  ...
}
```

Sandbox non configurata (default mode: "off"), profilo full. L'unico confine attivo è la tool policy, che — per sandbox-vs-tool-policy-vs-elevated.md:69 — non rende read-only nulla finché exec è permesso. Postura accettabile per un assistente personale, **inaccettabile come base di un deployment cliente**.

8. Verificabilità: cosa resta registrato

8.1 Chiarimento che evita un errore da contratto

Due comandi con nomi quasi identici fanno cose opposte:

Comando	Cosa fa
<code>openclaw audit</code>	Registro storico delle azioni: agent run e tool action, solo metadati (docs/cli/audit.md:11)
<code>openclaw security audit</code>	Analisi statica della config per foot-gun di sicurezza. Nessuno storico

Help letterale di `openclaw audit`:

Usage: `openclaw audit [options]`

Inspect metadata-only agent run and tool action records

Options:

```
--after <timestamp>  Include records at/after ISO time or Unix milliseconds
--agent <id>         Filter by agent id
--before <timestamp> Include records at/before ISO time or Unix milliseconds
--cursor <sequence>  Continue from a previous result cursor
--json              Output a bounded JSON page (default: false)
--kind <kind>        Filter by kind (agent_run or tool_action)
--limit <count>      Maximum records (1-500) (default: "100")
--run <id>           Filter by run id
--session <key>      Filter by exact session key
--status <status>    Filter by status (started, succeeded, failed, cancelled,
                    timed_out, blocked, unknown)
```

8.2 Il ledger: cosa contiene, cosa no, e quanto dura

Quattro azioni proiettate: `agent.run.started`, `agent.run.finished`, `tool.action.started`, `tool.action.finished` (docs/cli/audit.md:54-59). Ogni record porta redaction: `"metadata_only"`.

Non contiene (docs/cli/audit.md:15-18): prompt, messaggi, argomenti dei tool, risultati dei tool, output dei comandi, testo grezzo degli errori. I tool call id sono esportati solo come fingerprint one-way.

Schema reale della tabella, letto dal database:

```
CREATE TABLE audit_events (
  sequence INTEGER PRIMARY KEY AUTOINCREMENT,
  event_id TEXT NOT NULL UNIQUE,
  source_id TEXT NOT NULL UNIQUE,
  source_sequence INTEGER NOT NULL,
  occurred_at INTEGER NOT NULL,
  kind TEXT NOT NULL, action TEXT NOT NULL, status TEXT NOT NULL,
  error_code TEXT, actor_type TEXT NOT NULL, actor_id TEXT NOT NULL,
  agent_id TEXT NOT NULL, session_key TEXT, session_id TEXT,
  run_id TEXT NOT NULL, tool_call_id TEXT, tool_name TEXT
);
```

File: `~/openclaw/state/openclaw.sqlite` (91 MB + 4,5 MB di WAL). Contenuto misurato su questa macchina:

```
count | min(occurred_at) | max(occurred_at)
84253 | 1784541497722    | 1787132370779
```

La finestra è **29,98 giorni**: la ritenzione dichiarata di 30 giorni è confermata empiricamente. Ma 84.253 record su un cap di 100.000 significa che questa installazione è **all'84% del tetto**: se il traffico cresce, si perdono record **prima** dei 30 giorni.

8.3 Le sei superfici, e dove stanno

Superficie	Percorso	Contenuto
Audit ledger	~/.openclaw/state/op enclaw.sqlite -> audit_events	metadati azioni, no contenuto
Transcript sessione	~/.openclaw/agents/< id>/sessions/<sessio nId>.jsonl	messaggi, tool call e risultati
Log gateway	/tmp/openclaw/opencl aw-YYYY-MM-DD.log	JSONL, livello info
Trajectory sidecar	<session>.trajectory .jsonl o \$OPENCLAW_TR AJECTORY_DIR	timeline runtime
Scritture di config	~/.openclaw/logs/con fig-audit.jsonl	catena di hash
Comandi slash	~/.openclaw/logs/com mands.log + tabella command_log_entries	opt-in via hook command-logger

La scoperta più interessante è config-audit.jsonl, artefatto core (dist/io.audit-Dfj9UbCq.js). Prima riga reale:

```
{
  "ts": "2026-05-06T22:39:36.450Z",
  "source": "config-io",
  "event": "config.write",
  "configPath": "/Users/apple/.openclaw/openclaw.json",
  "pid": 9806,
  "ppid": 9804,
  "argv": [
    "...",
    "openclaw",
    "gateway",
    "install",
    "--force"
  ],
  "previousHash": "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855",
  "nextHash": "ec220c1137bfadeef557a6fb4e54ac61dba4e0712412d49ff0f8b160486aa79",
  ...
}
```

previousHash -> nextHash su ogni scrittura, con argv e pid. **È il pezzo più vicino a un registro a prova di manomissione che esista nel sistema — e riguarda solo la config.** Non è una firma: è un digest concatenato senza chiave. Chi può riscrivere il file può ricalcolare la catena.

Trappola di naming: openclaw transcripts **non** riguarda le conversazioni. È l'ispettore per l'audio di riunioni catturato dal tool transcripts (voce Discord), opt-in, default false. Su questa macchina la directory non esiste.

8.4 Trajectory: il più completo, e il più fragile

Per sessione, attivo di default (docs/tools/trajectory.md:11, :75). Bundle: manifest.json, events.jsonl, session-branch.json, metadata.json, artifacts.json, prompts.json, system-prompt.txt, tools.json.

Include input e output dei tool (:50-51), usage e prompt-cache (:19). Export:

```
openclaw sessions export-trajectory --session-key "agent:main:telegram:direct:123" --workspace .
```

Ma: finestra scorrevole con cap a **10 MiB** che scarta gli eventi vecchi (:182); i sidecar sono **rimossi quando la sessione è prunata o sfrattata** (:138-141); si spegne con OPENCLAW_TRAJECTORY=0 **senza lasciare traccia del fatto che sia stato spento** (:146); e la redazione è dichiaratamente parziale: «Redaction is best-effort and cannot know every application-specific secret» (:188-189). Posizionamento ufficiale (:171): «for support and debugging, not public posting».

8.5 OpenTelemetry: captureContent è l'unica via per esportare il contenuto

È un plugin, non un modulo core: openclaw plugins install clawhub:@openclaw/diagnostics-otel.

```
{
  "plugins": {
    "allow": ["diagnostics-otel"],
    "entries": { "diagnostics-otel": { "enabled": true } }
  },
  "diagnostics": {
    "enabled": true,
    "otel": {
      "enabled": true,
      "endpoint": "http://otel-collector:4318",
      "protocol": "http/protobuf",
      "serviceName": "openclaw-gateway",

```

```

    "traces": true, "metrics": true, "logs": true,
    "logsExporter": "otlp",
    "sampleRate": 1.0,
    "flushIntervalMs": 60000,
    "captureContent": {
      "enabled": true,
      "inputMessages": true,
      "outputMessages": true,
      "toolInputs": true,
      "toolOutputs": true,
      "systemPrompt": true,
      "toolDefinitions": true
    }
  }
}
}

```

Quattro trappole verificate:

1. `protocol` supporta **solo** `http/protobuf`. Qualunque altro valore, incluso `grpc`, **aborta l'intera sottoscrizione** con un warning — e ferma anche l'export dei log su `stdout` (`docs/gateway/opentelemetry.md:62`).
2. `traces` e `metrics` sono on quando `otel.enabled` è `true`; `logs` è **off** e va acceso a mano (`:73–77`).
3. `captureContent: true` (booleano) accende cinque sottochiavi e i body dei log, **ma non** `systemPrompt`, che va messo separatamente (`:160–162`). Le sottochiavi granulari **non** accendono mai i body dei log (`:135–136`).
4. `toolInputs` / `toolOutputs` sono catturati **solo per il runtime agent integrato**. Gli harness esterni — **Codex, Claude CLI** — **emettono `span tool.execution.*` senza payload di contenuto** (`:164–173`). È un buco di copertura da dichiarare per iscritto.

8.6 Prometheus: la chiave non esiste, ed è corretto

Non c'è nessuna chiave `prometheus` nello schema perché è un plugin: `diagnostics-prometheus`.

```

{
  "plugins": {
    "allow": ["diagnostics-prometheus"],
    "entries": { "diagnostics-prometheus": { "enabled": true } }
  },
  "diagnostics": { "enabled": true }
}

```

Endpoint `GET /api/diagnostics/prometheus, text/plain; version=0.0.4`. **Richiede scope operator: «There is no public unauthenticated mode»** (`docs/gateway/prometheus.md:241`).

Metriche notevoli (underscore, non punti — catalogo distinto da OTEL): `openclaw_run_completed_total`, `openclaw_model_tokens_total`, `openclaw_model_cost_usd_total`, `openclaw_tool_execution_total`, `openclaw_tool_execution_blocked_total`, `openclaw_session_stuck_total`, `openclaw_prometheus_series_dropped_total`.

Due limiti: cap a **2048 serie**, oltre il quale le nuove vengono **scartate** (`:161–163`); e stato **solo in memoria**, azzerato a ogni restart (`:247`). Prometheus non è un registro storico lato OpenClaw.

8.7 Il registro append-only che si può davvero costruire

L'architettura, usando solo meccanismi verificati:

1. **Indice azioni** — `audit_events`, già attivo, zero lavoro. Interrogabile da CLI e da RPC `audit.list` (scope `operator.read`).
2. **Contenuto** — `diagnostics.otel.captureContent.*` verso un collector OTLP esterno.
3. **Metriche** — `diagnostics-prometheus` o OTEL metrics.
4. **Modifiche di config** — `config-audit.jsonl`, già attivo, con catena di hash.
5. **Comandi utente** — `openclaw hooks enable command-logger`.
6. **Approfondimento su caso singolo** — `export-trajectory`.

Il collante è il `traceId`, presente come chiave top-level nei log JSONL, sugli span OTEL, e propagato al provider via `traceparent`.

Cosa NON è garantito — da mettere per iscritto prima della firma:

- **Immutabilità: no.** Nessuna superficie è WORM. `audit_events` è una tabella sqlite ordinaria, scrivibile da chi ha accesso al file.
- **Firma crittografica: no.** L'unica catena di hash copre solo le scritture di config, non le azioni degli agenti, e non ha chiave.
- **Anti-tampering: no.** I check `fs.state_dir.perms_world_writable` e `fs.config.perms_writable` proteggono dall'*altro* utente della macchina, non dal proprietario.
- **Completezza: no**, per sette ragioni verificate: ritenzione audit 30 giorni con cap 100.000 (già all'84%); log gateway prunati dopo **24 ore**; sidecar trajectory con finestra scorrevole di 10 MiB; transcript soggetti a pruning e eviction (sul disco si vedono già suffissi `.jsonl.reset.<ts>` e `.jsonl.deleted.<ts>`); metriche Prometheus azzerate a ogni restart; `captureContent` cieco sugli harness esterni; e il writer dell'audit è dichiarato «bounded background» senza garanzie di durabilità.
- **Chi può cancellare cosa:** chiunque abbia accesso in scrittura a `~/openclaw` — cioè l'utente che fa girare il Gateway — può alterare **qualsiasi** superficie. Non esiste separazione di privilegi tra chi scrive il registro e chi può modificarlo.

In una riga per il cliente: si può costruire un registro completo e ispezionabile, ma è un registro cooperativo, non forense. Prova cosa è successo a chi si fida dell'host; non prova nulla contro chi controlla l'host.

L'unico rimedio reale è spedire gli eventi fuori dalla macchina in tempo reale — OTLP push verso un collector di terze parti — e far diventare *quella* copia il riferimento contrattuale. Il push è la proprietà che conta: l'host non può ritirare quello che ha già spedito.

9. Esiste di meglio?

9.1 Dentro OpenClaw

Sub-agent — isolamento di sessione, non di sicurezza. Perdono sempre `gateway`, `agents_list`, `session_status`, `cron`; i leaf perdono anche `sessions_*`. Ma **l'auth non è isolata**: «The main agent's auth profiles are merged in as a **fallback**... Fully isolated auth per agent is not supported yet» (`docs/tools/subagents.md:462-465`). **Questo li esclude come confine di accesso ai dati.** In più l'announce è best-effort: «If the gateway restarts, pending "announce back" work is lost» (:646).

Task Flow — un registro durevole con optimistic concurrency, non un motore di workflow. La proprietà forte (`docs/automation/taskflow.md:29`): «Every mutation passes the flow's expected revision. A stale write is rejected as a revision conflict instead of clobbering newer state.» Persistito in `flow_runs` nel sqlite; il cancel è sticky attraverso i restart.

Ma: **non ha nessuna chiave di configurazione.** Ricerca esaustiva sullo schema: `taskflow O`, `task_flow O`, `flow_runs O`. Non è governabile da `openclaw.json`: si guida da codice plugin o dalla CLI `openclaw tasks flow`. La ritenzione è cablata a 7 giorni. E non c'è replay: solo `status + state JSON + revision`.

Code mode — la sola feature qui che riduce la superficie del modello **senza** perdere tracciabilità (`docs/reference/code-mode.md:770-777`): «Every nested tool call crosses the host bridge and re-enters OpenClaw, preserving: active agent id, session id and key, sender and channel context, sandbox policy, approval policy, plugin `before_tool_call` hooks, abort signal... Nested calls project into the transcript as real tool calls». E fallisce chiuso. Ma lo stato di run è in una mappa in-process **non persistita**: un restart perde ogni run sospeso. Postura autodichiarata: «Security posture: model code is hostile», «Stability: experimental».

Harness Codex — contiene l'unica logica in tutto OpenClaw che distingue esplicitamente «sicuro da riprovare» da «non sicuro perché ci sono stati effetti collaterali» (`docs/plugins/codex-harness.md:740-745`). Ma il default è pericoloso:

«Local stdio app-server sessions default to... approvalPolicy: "never"... and sandbox: "danger-full-access"» (:385–387), e il perimetro nativo è **opaco agli hook**: «it does not rewrite native tool arguments» (:839–841). Per un flusso di business è la scelta sbagliata.

9.2 Fuori da OpenClaw

Temporal — l'unico che garantisce il determinismo per contratto e lo verifica a runtime. «any time your Workflow code is executed it makes the same Workflow API calls in the same sequence, given the same input» (<<https://docs.temporal.io/workflow-definition>>); «If a generated Command doesn't match what it needs to in the existing Event History, then the Workflow Execution returns a non-deterministic error». Le Activity — dove va l'LLM — sono fuori dal replay path e il loro risultato è **registrato**. Event History è audit intrinseco, non opt-in. Limiti dichiarati: at-least-once, non exactly-once; 51.200 eventi / 50 MB per esecuzione; retention Cloud default 30 giorni.

LangGraph — persistenza sì, determinismo **esplicitamente negato**. Citazione decisiva (<<https://docs.langchain.com/oss/python/langgraph/use-time-travel>>): «Replay re-executes nodes—it doesn't just read from cache. LLM calls, API requests, and interrupts fire again and may return different results.» Gli interrupt rieseguono l'intero nodo e l'idempotenza è scaricata sull'utente.

BAML — la migliore ergonomia sull'output (Schema-Aligned Parsing, @assert / @check), ma il retry è il più debole: copre solo gli errori di **rete**, non i fallimenti di parsing (<<https://docs.boundaryml.com/ref/llm-client-strategies/retry-policy>>). Avvertenza: quel dominio si autodichiara **vo**, "the legacy DSL".

Pydantic AI — il migliore su output tipizzato più retry più telemetria: ModelRetry con budget per agent/run/tool, e tracing OTel nativo con include_content=False per la privacy. La durabilità è **delegata** a Temporal/DBOS/Prefect/Restate.

DSPy — ottimizza i prompt, non l'affidabilità di esecuzione. Le Assertion sono deprecated: «Assertions are deprecated and NOT supported» (<<https://dspy.ai/learn/programming/7-assertions/>>), e il sostituto dspy.Refine restituisce il tentativo migliore invece di sollevare.

Architettura "niente agente" — API applicativa, query parametrizzate, LLM solo per la prosa finale. Non ha documentazione di vendor perché non è un prodotto. La proprietà decisiva è strutturale: **se l'LLM non sceglie né i dati né le azioni, non può inventarli né superare i permessi**. Spariscono per costruzione: selezione errata del tool, argomenti allucinati, escalation via prompt injection, non-determinismo dell'orchestrazione. Resta una sola classe di guasto — la parafrasi infedele — mitigabile con un controllo deterministico: ogni numero citato deve comparire nel JSON di input.

9.3 Verdetto

Sì, esiste una via più professionale, e non è un prodotto: è una separazione.

Il limite di «OpenClaw + MCP + hooks» non è la qualità dell'implementazione. È che **il piano di controllo e il piano di esecuzione sono lo stesso oggetto, e quell'oggetto è un LLM**. Gli hook sono l'unico punto di enforcement, e before_agent_finalize — il candidato naturale — è fail-open in sei modi (§4.2). Un controllo che in caso di guasto lascia passare non è un controllo.

Proprietà	OpenClaw + MCP + hooks	Alternativa superiore
Replay deterministico	assente	Temporal
Rilevamento di divergenza	assente	Temporal (Command/History mismatch)
Effetti collaterali confinati	nessun confine: l'hook si arrende se sono già avvenuti	Temporal (Activity fuori dal replay path)
Tipizzazione forzata	llm-task: un tentativo, nessun retry	Pydantic AI (ModelRetry) + BAML (@assert)

Proprietà	OpenClaw + MCP + hooks	Alternativa superiore
Audit firmato / WORM	nessuno	nessuno neanche negli altri
Controllo accesso dati	auth non isolata fra agenti	API applicativa con RBAC prima dell'LLM
Confine multi-tenant	escluso dalla doc stessa	qualunque API applicativa
Maturità dichiarata	68% — Alpha , coverage «Experimental 4%»	—

Il dato di maturità è autodichiarato e verificabile (`docs/maturity/scorecard.md:22-32`): 68% complessivo, Quality Alpha 64%, Completeness Beta 71%, Coverage Experimental 4%. È il numero più difficile da discutere in una sala riunioni, perché non lo dice un concorrente.

Chiarimento anti-slogan: nessuno dei sistemi confrontati rende deterministico l'LLM. Temporal ottiene la riproducibilità **registrando** il risultato dell'Activity, non ri-derivandolo. Chi promette «agenti deterministici» sta vendendo determinismo *dell'orchestrazione* e lo spaccia per determinismo *del modello*.

Dove OpenClaw vince davvero — e sono cose che gli altri cinque non fanno affatto: multicanale con thread binding persistente (Telegram, Discord, Slack, Matrix, iMessage, WhatsApp); tempo di consegna (un flusso schedulato multicanale è un comando cron); operatività (`openclaw doctor --fix, tasks audit, export-trajectory, maintenance sweep` — è pensato per essere gestito da una persona sola); costo (zero infrastruttura server); e una riduzione della superficie tool fatta bene.

9.4 Architettura ibrida raccomandata

La regola in una riga: OpenClaw è il canale e il redattore. Non è il decisore né l'esecutore.

```

Utente -> canale OpenClaw (Telegram / Slack / ...)
| estrazione intent in JSON tipizzato
v
[CONFINE] <- qui l'LLM smette di decidere
|
+- API applicativa: authZ dell'utente reale, query parametrizzate,
| log applicativo (l'unico audit che regge in causa)
|
+- Temporal, solo se il flusso è lungo / multi-step / con scritture:
| workflow deterministico + Activity per ogni effetto collaterale
|
+- risultato: JSON con provenienza obbligatoria (sourceUrl, retrievedAt, asOf)
v
LLM finale = SOLO formulazione della prosa, input esclusivamente il JSON approvato
| output tipizzato: Pydantic AI (ModelRetry) o BAML (@assert)
v
Controllo deterministico post-generazione: ogni cifra citata deve comparire nel JSON
v
message_sending {cancel:true} = ULTIMA rete, non la prima
v
Consegna multicanale OpenClaw

```

L'LLM sta **prima** della decisione (estrazione intent: riparabile, basta richiedere) e **dopo** l'esecuzione (redazione: verificabile numero per numero). Nel mezzo, dove si accede ai dati e si producono effetti, non c'è modello.

Questo pattern è già raccomandato dentro OpenClaw stesso (`docs/automation/taskflow.md:143-155`): campi di provenienza obbligatori, scarto degli item stantii **prima** della sintesi, e «The LLM step should receive only structured JSON».

Cosa tenere di OpenClaw: code mode se il catalogo tool è grande; Task Flow per tracciare l'esecuzione attraverso i restart; trajectory con `OPENCLAW_TRAJECTORY_DIR` puntato **fuori** dalla directory delle sessioni, così la maintenance non lo cancella, e spedito a un sink esterno.

Cosa non tenere: i sub-agent come confine di credenziali; l'harness Codex per flussi di business; `llm-task` per un passo che deve riuscire; `before_agent_finalize` come controllo di conformità.

Quando l'ibrido non serve: se il cliente vuole solo «chiedo in Slack, ottengo un dato dai miei sistemi», salta Temporal. API più query parametrizzate più Pydantic AI per la prosa, OpenClaw come canale. Temporal si aggiunge quando compare la prima scrittura, o il primo flusso che deve sopravvivere a un crash a metà.

10. Rischi: cosa può rompersi al prossimo aggiornamento

10.1 Nota di metodo

Le prime query all'API GitHub anonima hanno restituito **HTTP 403** («API rate limit exceeded»). Tutte le ricerche sono state rifatte con `gh api` autenticato. I totali dei termini larghi (`hook` 656, `lobster` 627, `sandbox` 425, `mcp` 243, `llm-task` 110) sono match full-text, **non** conteggi di issue tematiche — `lobster` compare nella descrizione del repo. I termini stretti sono significativi: `before_agent_finalize` 9, `determinism` 6, `temperature` 15, `message_sending` 26.

Disallineamento di versione da tenere presente: il pacchetto locale è `2026.7.1-2` con file `dist/` datati **18 luglio 2026**, mentre le issue arrivano al **18 agosto 2026**. L'installazione è circa un mese dietro HEAD: alcune di queste potrebbero essere già corrette upstream.

10.2 Il gate d'uscita non è garantito — blocker assoluto

#	Issue	Data	Perché conta
125802	[Bug]: successful provider streams can skip response hooks	18 ago 2026	Un gate fail-open che in più può non essere invocato affatto
13583	[Feature] Pre-response enforcement hooks (hard gates) for mandatory tool-call / policy rules	10 feb 2026	I hard gate pre-risposta non esistono : sono una richiesta aperta da sei mesi
119826	[Feature]: Introduce an atomic <code>before_assistant_message_commit</code> lifecycle boundary	6 ago 2026	Un confine atomico di commit del messaggio è proposto, non implementato
56349	[Feature]: Unbypassable outbound policy enforcement (pre-send guarantee)	28 mar 2026	Se è aperta, oggi l'enforcement outbound è bypassabile
25592	Text between tool calls leaks to messaging channels	24 feb 2026	Conferma indipendente: testo che raggiunge il canale fuori dal percorso hook

<<https://github.com/openclaw/openclaw/issues/125802>> · <<https://github.com/openclaw/openclaw/issues/13583>> · <<https://github.com/openclaw/openclaw/issues/119826>> · <<https://github.com/openclaw/openclaw/issues/56349>> · <<https://github.com/openclaw/openclaw/issues/25592>>

10.3 La premessa «le prove dei tool sono reali» non regge

#	Issue	Data
45049	Agent loop allows simulated tool calls instead of enforcing real tool invocation	13 mar 2026
49876	Cron sessions deliver hallucinated output instead of failing cleanly when tool calls fail	18 mar 2026

#	Issue	Data
125449	Exec-failure banner is too easy to miss when assistant still asserts confident (unsupported) numbers in the same message	17 ago 2026

La #45049 è particolarmente insidiosa per il progetto del cancello: **un verificatore che confronta la risposta contro "le prove dei tool" può confrontarla contro prove che il modello si è inventato**. La #49876 riguarda direttamente i cron automatici.

10.4 Deprecazioni con data — due sono già scadute

Superficie	Rimozione	Fonte
deactivate (alias di gateway_stop)	2026-08-16 — scaduta da 3 giorni	docs/plugins/hooks.md:608-609
api.runtime.tasks.flow -> managedFlows	2026-07-26 — scaduta da 24 giorni	docs/plugins/sdk-migration.md:822
subagent_spawning	2026-08-30 — fra 11 giorni	docs/plugins/hooks.md:154, :598-601
Envelope in chiaro in inbound_claim / message_received -> BodyForAgent	«next major release»	docs/plugins/hooks.md:596-600
before_agent_start -> before_model_resolve + before_prompt_build	nessuna data	docs/plugins/hooks.md:602-604
onResolution in before_tool_call: da string libera a union tipata	già cambiato	docs/plugins/hooks.md:612-615
api.registerSessionExtension -> api.session.state...	nessuna data	docs/plugins/hooks.md:480, :616-619
api.enqueueNextTurnInjection -> api.session.workflow...	nessuna data	docs/plugins/hooks.md:485

Politica dichiarata (docs/plugins/sdk-migration.md:971-976): a ogni removeAfter scaduto la superficie diventa *eligible for removal* e la CI upstream fallisce con --fail-on-eligible-compat. «Next major release: any surfaces still not migrated are removed; plugins still using them will fail.»

10.5 MCP: rottura annunciata

#119294 — «Track migration to MCP 2.0 (2026-07-28 stateless spec) — @modelcontextprotocol/sdk -> split client/server v2 packages» (4 ago 2026). Cambio di SDK e di modello di sessione: **qualunque integrazione MCP fatta oggi va rifatta**.

Più tre guasti che riportano verde mentre sono morti:

- #122712 (12 ago) — server aggiunti con openclaw mcp add non raggiungono mai la sessione claude-cli, nonostante doctor/probe sani
- #114154 (26 lug) — bundle-mcp passa la policy e il probe è sano, ma le sessioni non lo montano mai
- #111930 (20 lug) — registrazione MCP **solo globale**, nessuno scoping per agente: in un contesto multiutente non è isolabile

10.6 Sandbox: il verde mente, con label P1

- **#116053** (29 lug, **P1**) — «Sandbox: MCP tools never reach sandboxed sessions despite `tools.sandbox.tools.alsoAllow` (and doctor reports a false all-clear)»
- **#84942** (21 mag) — la policy dice `runtime=sandboxed` mentre il runner Claude CLI disabilita la sandbox
- **#124797** (16 ago) — il policy check ignora `tools.exec.mode` e classifica agenti ristretti come `exec security full`

Tre modi diversi per cui un cruscotto verde non corrisponde alla realtà. Per un deployment cliente questo significa una cosa sola: **il verde va verificato dal vivo, non letto.**

10.7 Identità e attribuzione

- **#69110** (19 apr) — «Gateway-level model identity not passed to delivery hooks (enables silent model-tag forgery)»: un hook di consegna non può fidarsi di quale modello ha prodotto il testo
- **#112677** (22 lug) — `llm-task` embedded Codex **perde l'identità dell'agente chiamante** e risolve `tools.exec` dalla policy **globale**: escalation di privilegi di fatto in una fleet multiagente

10.8 Il CHANGELOG non aiuta

```
$ wc -l /Users/apple/.local/lib/node_modules/openclaw/CHANGELOG.md
9
```

Il file intero, nove righe, una sola voce relativa alla release corrente. **Il tarball npm non spedisce storico.** Nessuna rinomina di hook, nessun breaking change, nessun cambio di default è desumibile da lì. Lo storico va cercato su GitHub Releases.

10.9 Le tre cose da mettere in contratto

1. **Pinnare la versione.** Con `deactivate` già scaduto e `subagent_spawning` in scadenza fra 11 giorni, un `npm update` non pilotato può rompere un plugin in produzione senza preavviso.
2. **Un banco di regressione che preme i tasti.** Non un elenco di feature: un test che manda un turno vero, verifica che il cancello blocchi, e fallisce se passa. Le issue **#116053** e **#84942** esistono proprio perché il cruscotto diceva verde.
3. **Trattare MCP come superficie a rottura annunciata.** L'integrazione va scritta dietro un adattatore proprio, non sparsa nella config.

(b) Quale livello impedisce quale errore

Legenda: [SI] impedisce · [!] mitiga ma non garantisce · [NO] non copre

Livello	Invenzione	Dato mancante	Query sbagliata	Scrittura non autorizzata	Lentezza
Server MCP con operazioni parametrizzate	[!] toglie al modello la possibilità di comporre SQL	[SI] l'operazione o restituisce dati o errore esplicito	[SI] il modello non scrive query: sceglie un'operazione e passa parametri	[SI] il server espone solo letture	[!] timeout dichiarativi, ma il <code>tools/list</code> può costare fino a 30 s se lo configuri male
<code>toolFilter</code> (per server)	[NO]	[NO]	[SI] un'operazione fuori lista non entra nel catalogo	[SI] <code>run_sql</code> non arriva mai al modello	[SI] meno tool = meno schema nel prompt
<code>compat.supportsStrictMode</code>	[NO] inefficace fuori da OpenAI/Azure ; e nessun modello lo dichiara <code>true</code> qui	[NO]	[!] solo su rotta nativa OpenAI, e con downgrade silenzioso	[NO]	[NO]
<code>tools.deny / profile</code>	[NO]	[NO]	[NO]	[SI] hard stop : <code>/exec</code> non scavalca un <code>exec</code> negato	[SI] meno schemi nel prompt
<code>tools.exec.mode: "deny"</code>	[NO]	[NO]	[NO]	[SI] blocca l'host <code>exec</code> (usare insieme al <code>deny</code> di policy)	[NO]
Sandbox	[NO]	[NO]	[NO]	[!] «not a perfect security boundary»; i <code>bind mount</code> la bucano	[NO] aggiunge overhead
Exec approvals	[NO]	[NO]	[NO]	[!] «not a per-user auth boundary»	[NO] peggiora: attesa umana
<code>before_tool_call</code>	[NO]	[NO]	[SI] può riscrivere i parametri o bloccare	[SI] può imporre approvazione	[NO]
<code>after_tool_call</code>	[NO] osserva soltanto	[SI] rileva l'assenza di prove (poi serve un altro livello per agire)	[NO]	[NO]	[NO]
<code>llm-task</code> con schema	[!] vincola la forma , non la verità; un tentativo solo	[SI] campo assente = errore di validazione	[NO]	[SI] tool disabilitati nel run	[!] timeout 30 s mascherato da errore di formato
<code>before_agent_finalize</code>	[!] fail-open in sei modi : correttivo, non bloccante	[!] stessa riserva	[NO]	[NO]	[NO] peggiora: fino a 3 passate extra
<code>message_sending {cancel:true}</code>	[SI] unico blocco duro dell'uscita	[SI] idem	[NO]	[NO] arriva dopo gli effetti collaterali	[NO]
<code>reply_payload_sending</code>	[SI] come sopra, e porta <code>runId</code> nel <code>ctx</code>	[SI]	[NO]	[NO]	[NO]

Livello	Invenzione	Dato mancante	Query sbagliata	Scrittura non autorizzata	Lentezza
Controllo deterministico post-generazione (ogni cifra deve comparire nell'input)	[SI] l'unico che chiude la classe di guasto	[SI]	[NO]	[NO]	[SI] costo zero, nessuna chiamata al modello
<code>thinkingDefault: "off"</code>	[NO]	[NO]	[NO]	[NO]	[SI] l'unica leva reale su DeepSeek V4
Sfoltire skill e tool	[NO]	[NO]	[NO]	[!] meno superficie	[SI] 12.437 + 31.833 caratteri misurati
<code>contextInjection: "continuation-skip"</code>	[NO]	[NO]	[NO]	[NO]	[SI]
<code>contextPruning: "cache-ttl"</code>	[NO]	[NO]	[NO]	[NO]	[SI]
Tool Search	[NO]	[NO]	[NO]	[NO]	[!] può peggiorare su cataloghi piccoli
Code mode	[NO]	[NO]	[NO]	[!] preserva policy e hook sulle nested call	[SI] meno round-trip
Audit ledger / OTEL	[NO] non impedisce: registra	[NO]	[NO]	[NO]	[NO]
API applicativa fuori dall'LLM	[SI] chiude la classe alla radice	[SI]	[SI]	[SI] RBAC dell'utente reale	[SI]

Lettura in una riga: nessuna riga di questa tabella ha tutte le colonne verdi tranne l'ultima. Il livello che impedisce l'invenzione in modo strutturale è quello che toglie all'LLM la decisione, non quello che la controlla dopo.

(c) Cosa con OpenClaw oggi NON è ottenibile

Elenco onesto. Ogni voce è una cosa che il cliente potrebbe chiedere e che va risposta con un no.

1. **Un cancello anti-invenzione che non si possa saltare.** `before_agent_finalize` è fail-open in sei modi verificati nel codice, più un settimo aperto come issue (#125802). `message_sending {cancel:true}` blocca davvero, ma arriva **dopo** che gli eventuali effetti collaterali sono avvenuti. Un pre-send garantito è una feature request aperta da marzo (#56349).
2. **La certezza che le "prove dei tool" siano reali.** #45049: il loop dell'agente consente tool call simulate invece di imporre l'invocazione reale. Un verificatore può confrontare la risposta contro prove inventate.
3. **Un audit forense.** Nessuna superficie è WORM, firmata o append-only in senso forte. L'unica catena di hash copre le scritture di config, non le azioni. Chi fa girare il Gateway può alterare tutto. La trajectory si spegne con una variabile d'ambiente senza lasciare traccia.
4. **Completezza dello storico oltre 30 giorni.** Audit ledger: 30 giorni e 100.000 record, già all'84% su questa macchina. Log gateway: **24 ore**. Trajectory: finestra scorrevole di 10 MiB. Metriche Prometheus: azzerate a ogni restart.
5. **Isolamento multi-tenant.** Dichiarato fuori scopo dal progetto stesso. Non esistono identità utente distinte con credenziali separate, autorizzazione per utente, rate limiting per mittente, né isolamento dati per utente. Chi ha il token operatore legge tutte le sessioni e tutta la config. Per più clienti servono più Gateway su utenti di sistema separati.
6. **Isolamento delle credenziali fra agenti.** «Fully isolated auth per agent is not supported yet» (`docs/tools/subagents.md:462-465`). I sub-agent non sono un confine di accesso ai dati.
7. **Replay deterministico e rilevamento di divergenza.** Non esiste nulla di paragonabile all'Event History di Temporal. Task Flow ha il revision CAS, ma nessun event log e nessuna ricostruzione dello stato.
8. **Structured output garantito.** `response_format` è un pass-through non validato. `compat.supportsStrictMode` non produce `strict:true` fuori da OpenAI/Azure, e nessun modello lo dichiara `true` in questa installazione.
9. **Un `llm-task` che ritenta.** Un tentativo, nessun repair-loop. E il timeout è mascherato da "LLM returned empty output".
10. **Validazione dei parametri di campionamento.** `temperature` vive in un contenitore free-form. Un valore stringa viene ignorato in silenzio; un valore fuori range non viene fermato. `compat.supportsTemperature` è un flag morto.
11. **Prova che un parametro sia arrivato al provider, usando `--raw-stream`.** Quel flag registra solo il lato risposta. Serve `openclaw proxy`.
12. **Configurabilità di Task Flow.** Zero chiavi nello schema. Ritenzione cablata a 7 giorni.
13. **Un cruscotto di cui fidarsi senza verifica.** Tre issue aperte descrivono `doctor/probe/policy check` che riportano verde su configurazioni rotte (#116053 P1, #84942, #124797).
14. **Copertura del contenuto sugli harness esterni.** `captureContent` non cattura input e output dei tool per Codex e Claude CLI.
15. **Stabilità delle superfici plugin.** Due deprecazioni con data **già scaduta**, una in scadenza fra 11 giorni, e una migrazione MCP 2.0 annunciata che rifà l'SDK.

(d) Registro di lavoro

d.1 Metodo, e un incidente da dichiarare

La ricerca è stata condotta con otto indagini parallele su aree distinte, coordinate da questa sessione. **Sette delle otto sono morte** quando il processo che le ospitava è uscito, e sono state rilanciate in primo piano. Una di esse, prima di essere corretta, aveva annotato numeri di issue GitHub **non ancora verificati** (#13583, #119826, #45049, #125449, #125802): le query sono state rieseguite integralmente e quei numeri risultano ora tutti confermati da output letterale — ma l'episodio va dichiarato, perché è esattamente il tipo di guasto che questo documento cerca di intercettare.

Un secondo costo da dichiarare: durante la ricerca sulla latenza è stato speso **un turno reale a pagamento** su `opencode-go/deepseek-v4-flash`, nel tentativo di eseguire `/context list` da CLI. Il comando **non viene intercettato** su quel percorso ed è finito al modello, che ha risposto a prosa inventata. I numeri della sezione 5.1 **non** vengono da lì: vengono dal `systemPromptReport` persistito nel session store.

d.2 Comandi eseguiti, con output grezzo

Versione — punto di partenza obbligato:

```
$ openclaw --version
OpenClaw 2026.7.1-2 (0790d9f)
```

Schema di configurazione — l'autorità usata per ogni chiave citata:

```
$ openclaw config schema > /tmp/oc-schema.json
exit=0
-rw-----@ 1 apple wheel 2519129 Aug 19 11:13 /tmp/oc-schema.json

top keys: ['$schema', 'accessGroups', 'acp', 'agents', 'approvals', 'audit', 'auth',
'bindings', 'broadcast', 'browser', 'channels', 'cli', 'commands', 'commitments',
'crestodian', 'cron', 'diagnostics', 'discovery', 'env', 'gateway', 'hooks', 'logging',
'marketplaces', 'mcp', 'media', 'memory', 'messages', 'meta', 'models', 'nodeHost',
'plugins', 'proxy', 'secrets', 'security', 'session', 'skills', 'surfaces', 'talk',
'tools', 'transcripts', 'tui', 'ui', 'update', 'web', 'wizard']
```

8.516 percorsi estratti e indicizzati. Ogni chiave citata nel documento è stata cercata qui prima di essere scritta.

Esistenza delle pagine richieste — verifica una per una:

OK gateway/configuration-reference.md	OK tools/thinking.md
OK gateway/config-agents.md	OK tools/subagents.md
OK gateway/config-tools.md	OK tools/skills.md
OK tools/index.md	OK automation/taskflow.md
MISS tools/mcp	MISS gateway/multi-tenant-hosting
OK cli/mcp.md	OK gateway/opentelemetry.md
OK tools/llm-task.md	OK gateway/prometheus.md
OK tools/lobster.md	OK reference/prompt-caching.md
OK plugins/hooks.md	OK providers/deepseek.md
OK automation/hooks.md	OK security/THREAT-MODEL-ATLAS.md
OK plugins/tool-plugins.md	OK tools/exec-approvals.md
OK plugins/building-plugins.md	OK tools/permission-modes.md
OK gateway/sandboxing.md	OK tools/elevated.md
DIR gateway/security	OK gateway/security/audit-checks.md
MISS gateway/audit	OK cli/audit.md
OK tools/trajectory.md	OK tools/loop-detection.md

Redirect dichiarati, per escludere che le tre mancanti fossero solo rinominate:

```
n redirects 238
{'source': '/mcp', 'destination': '/cli/mcp'}
{'source': '/webhook', 'destination': '/automation/cron-jobs'}
{'source': '/automation/webhook', 'destination': '/automation/cron-jobs'}
```

Nessun redirect per `/tools/mcp`, `/gateway/audit`, `/gateway/multi-tenant-hosting`.

Blocco oauth di MCP — la prova che **identity** non esiste:

```
{
  "type": "object",
  "properties": {
    "scope": { "type": "string", "minLength": 1, "title": "MCP OAuth Scope" },
    "redirectUrl": { "type": "string", "format": "uri", "title": "MCP OAuth Redirect URL" },
    "clientMetadataUrl": { "type": "string", "format": "uri", "title": "MCP OAuth Client Metadata URL" }
  },
  "additionalProperties": false,
  "title": "MCP OAuth"
}
```

Ricerca di allowConversationAccess nello schema — 75 occorrenze, tutte sotto `plugins.entries.<id>.hooks.`, nessuna a livello `hooks.`:

```
.plugins.entries.acpx.hooks.allowConversationAccess
.plugins.entries.active-memory.hooks.allowConversationAccess
.plugins.entries.anthropic.hooks.allowConversationAccess
... (75 in totale, una per plugin dichiarato)
```

Chiavi cercate e NON trovate nello schema:

```
trajectory: 0      promptCaching: 0      telemetry: 0
prometheus: 0      llmTask: 0        lobster: 0
permissionMode: 0  temperature: 0    taskflow: 0
```

compat.* — dove stanno davvero:

```
supportsStrictMode:  models.providers.<*>.models[].compat.supportsStrictMode  (1 percorso)
supportsTemperature: models.providers.<*>.models[].compat.supportsTemperature  (1 percorso)
```

before_agent_finalize nel codice compilato:

```
dist/embedded-agent-DGUuxGR2.js:4469  log warn: revision requested
dist/hook-runner-global-BmIrGllG.js:257  before_agent_finalize: 15e3
dist/lifecycle-hook-helpers-BwL6869q.js:114,122  hasHooks / catch -> continue
dist/selection-JInn13lc.js:13318,13334,13372  entry gate / revision limit / side effects
dist/schema-DRy01XBt.js:819  descrizione di allowConversationAccess
dist/provider-capabilities-CYpG67go.js:3739  before_agent_finalize: "Stop" (relay Codex)
```

```
$ grep -rno "MAX_BEFORE_AGENT_FINALIZE_REVISIONS *=[0-9]*" dist/*.js
dist/embedded-agent-DGUuxGR2.js:1774:MAX_BEFORE_AGENT_FINALIZE_REVISIONS = 3
```

```
$ grep -n "hadDeterministicSideEffect" dist/*.js | head -5
dist/selection-JInn13lc.js:2007
dist/selection-JInn13lc.js:3914
dist/selection-JInn13lc.js:4055
dist/selection-JInn13lc.js:4361
dist/selection-JInn13lc.js:4993
```

message_sending cancel:

```
$ grep -rno "cancelled_by_message_sending_hook" dist/*.js
dist/deliver-BdKtkX_b.js:1357
```

MCP — stato reale:

```
$ openclaw mcp status
MCP server status (/Users/apple/.openclaw/openclaw.json):
- meta-dev: disabled oauth
- n8n: disabled
- notes-reminders: disabled
- paperclip-board: disabled
- shortcuts: disabled
- sqlite: disabled
- tracking: disabled
- trenitalia: disabled
```

Modelli che dichiarano supportsStrictMode: true:

```
$ grep -rn '"supportsStrictMode": *true|supportsStrictMode: *true' \
  /Users/apple/.local/lib/node_modules/openclaw /Users/apple/.openclaw
(nessun risultato)
```

Plugin installati:

```
Plugins (30/74 enabled)
Source roots:
  stock: /Users/apple/.local/lib/node_modules/openclaw/dist/extensions
  global: /Users/apple/.openclaw/extensions

| LLM Task | llm-task | openclaw | disabled | stock:llm-task/index.js | 2026.7.1 |

$ openclaw plugins list 2>&1 | grep -ci "lobster"
0
$ ls dist/extensions | grep -i "lobster\\|llm"
litellm
llm-task
vllm
```

Il plugin che risolve il problema, trovato dal vivo:

```
$ openclaw plugins search llm-grounded
ClawHub plugins (1)
llm-grounded code-plugin | community | v0.2.4 – Deterministic grounding and voice contract
for LLM agents. Framework-independent core, with an OpenClaw plugin adapter included.
  Install: openclaw plugins install clawhub:llm-grounded
```

Flag del gateway:

```
--raw-stream          Log raw model stream events to jsonl (default: false)
--raw-stream-path <path> Raw stream jsonl path
```

Audit ledger — contenuto reale:

```
count | min(occurred_at) | max(occurred_at)
84253 | 1784541497722    | 1787132370779

agent_run|blocked|2      tool_action|blocked|17
agent_run|cancelled|184  tool_action|cancelled|17
agent_run|failed|101     tool_action|failed|2227
agent_run|started|2632   tool_action|started|39449
agent_run|succeeded|2431 tool_action|succeeded|36887
agent_run|timed_out|25   tool_action|timed_out|281

$ openclaw audit --limit 2
TIME      KIND      STATUS      AGENT      RUN      ACTION
2026-08-19T09:39:46.458Z tool_action succeeded    main      8ae7b5bb-42b2-403...
  tool.action.finished:Bash
```

Config reale (segreti oscurati):

```
$ openclaw config get agents.defaults.sandbox
Config path not found: agents.defaults.sandbox. Run openclaw config validate to inspect config shape.

$ openclaw config get logging
Config path not found: logging. Run openclaw config validate to inspect config shape.

$ openclaw config get diagnostics
{ "stuckSessionWarnMs": 30000, "stuckSessionAbortMs": 120000 }

$ openclaw config get tools | head -6
{ "profile": "full", "deny": ["video_generate", "image_generate"], ... }

$ openclaw config get hooks
{ "internal": { "enabled": true, "entries": { "boot-md": {...}, "command-logger": {...}, ... } } }
```

CHANGELOG:

```
$ wc -l CHANGELOG.md
9
```

Toolchain PDF disponibile su questa macchina:

pandoc	OK	pdflatex	-
wkhtmltopdf	-	xelatex	-
weasyprint	-	lualatex	-
prince	-	tectonic	-
typst	-		
reportlab 5.0.0			

PDF prodotto con `pandoc` (markdown -> HTML) più Brave in modalità headless (`--print-to-pdf`), con profilo temporaneo isolato per non toccare quello dell'utente.

d.3 Pagine consultate, una per una

Lette per intero: docs/tools/lobster.md (380 righe), docs/tools/llm-task.md (143), docs/plugins/hooks.md (630, con approfondimento su 18-90, 85-215, 297-506), docs/gateway/sandboxing.md, docs/gateway/sandbox-vs-tool-policy-vs-elevated.md, docs/cli/audit.md (93), docs/gateway/security/audit-checks.md (153), docs/tools/trajectory.md (214), docs/gateway/opentelemetry.md (472), docs/gateway/prometheus.md (256), docs/reference/prompt-caching.md (274), docs/providers/deepseek.md (147), docs/tools/thinking.md (146), docs/tools/tool-search.md, docs/reference/code-mode.md, docs/cli/mcp.md, docs/gateway/security/index.md, docs/security/THREAT-MODEL-ATLAS.md, docs/tools/subagents.md, docs/automation/taskflow.md, docs/plugins/codex-harness.md, docs/maturity/scorecard.md.

Lette per sezione: docs/gateway/configuration-reference.md, docs/gateway/config-tools.md, docs/gateway/config-agents.md, docs/automation/hooks.md, docs/plugins/tool-plugins.md, docs/plugins/building-plugins.md, docs/plugins/manifest.md, docs/plugins/sdk-migration.md, docs/tools/exec-approvals.md, docs/tools/exec-approvals-advanced.md, docs/tools/permission-modes.md, docs/tools/elevated.md, docs/tools/loop-detection.md, docs/tools/skills.md, docs/gateway/logging.md, docs/logging.md, docs/gateway/operator-scopes.md, docs/gateway/authentication.md, docs/cli/transcripts.md, docs/concepts/session.md, docs/concepts/context.md, docs/tools/multi-agent-sandbox-tools.md, docs/tools/clawhub.md, docs/automation/clawflow.md, docs/automation/tasks.md, docs/plugins/codex-harness-reference.md, docs/tools/code-execution.md.

Verificato nel codice compilato (non solo letto nei doc): dist/zod-schema-09m1_nmo.js, dist/schema-DRy01XBt.js, dist/agent-bundle-mcp-runtime--G82BMQs.js, dist/agent-bundle-mcp-names-B9PLR-i_.js, dist/agent-bundle-mcp-materialize-D9l-gQ5S.js, dist/agent-bundle-mcp-filter-DA8pW_vZ.js, dist/tool-policy-pipeline-C3ed0W1F.js, dist/agent-tools.policy-YD9HuYg0.js, dist/sender-tool-policy-BAIGCLiF.js, dist/sandbox-tool-policy-CLB7s2K0.js, dist/local-roots-CAoJyC6u.js, dist/group-policy-BiTZkkCF.js, dist/hook-types-DQ9eTy2x.d.ts, dist/hook-runner-global-BmIrGllG.js, dist/lifecycle-hook-helpers-BwL6869q.js, dist/registry-B8eQDFB4.js, dist/selection-JInn13lc.js, dist/embedded-agent-DGUuxGR2.js, dist/dispatch-DnzGTpPs.js, dist/deliver-BdKtkX_b.js, dist/get-reply-0TG64ybi.js, dist/hook-helpers-GgBV1rQp.js, dist/extra-params-cce1g0up.js, dist/openai-transport-stream-B0WkSqXp.js, dist/provider-stream-CqSx37Uy.js, dist/provider-stream-shared-B4Hm1tKd.js, dist/stream-BcRkg2P0.js, dist/openai-completions-compat-D9SGcXbn.js, dist/moonshot-thinking-y4RaKUa.js, dist/prepare.runtime-CM3Uj6Uh.js, dist/schema-validator-BRkrm3P2.js, dist/extensions/llm-task/index.js, dist/run-CSvF3r1e.js, dist/io.audit-Dfj9UbCq.js, dist/official-external-plugin-catalog-ph3rbXr3.js, dist/sandbox-cli-DhN0VhUT.js, dist/cli-backend-C4iY7FFY.js, dist/cli-backend-Dym4FoF1.js, più node_modules/@openclaw/ai/dist/openai-responses-shared-sj2YUPYc.mjs.

Letto ma non installato: il tarball npm @openclaw/lobster@2026.7.1, scaricato in sola lettura con npm pack in /tmp/lob. Tutte le righe di Lobster in §3.4 e §3.5 si riferiscono a quel pacchetto.

d.4 Ricerche che non hanno dato risultato

- curl **anonimo** su api.github.com -> **HTTP 403** dopo circa dieci richieste ("API rate limit exceeded for 37.183.79.66"). Tutte le query rifatte con gh api autenticato.

- `grep "removal"` su `docs/gateway/configuration-reference.md` -> **zero occorrenze**. Le sole voci deprecate lì sono il webhook legacy dei cron, **senza data**.
- **CHANGELOG: impossibile leggere le "ultime 300 righe"** — il file ne ha nove.
- **npm come registro dei plugin di verifica: strada sbagliata.** `llm-grounded`, `vericlaw`, `openclaw-accountability`, `proofrail-openclaw`, `agent-audit-gate`, `critiqor-openclaw`, `@laninthalesdran/openclaw-reliability-engine` -> tutti **HTTP 404** su `registry.npmjs.org`. Sono pacchetti ClawHub-nativi. L'unico su npm è `@traceproof/openclaw-traceproof-runtime`.
- `registry.npmjs.org/-/v1/search?text=scope:openclaw` -> `"total": 0`. npm non supporta quella sintassi di scope.
- **Le ricerche npm multi-parola non producono segnale:** `openclaw hallucination`, `openclaw grounding`, `openclaw verify`, `openclaw factcheck` restituiscono tutte gli stessi pacchetti `@openclaw/*` generici, con totali fra 9.886 e 49.384. La ricerca testuale npm è OR-fuzzy e ordina per popolarità.
- **GitHub code search** `before_agent_finalize` -> 1.996 risultati, quasi tutti fork o copie vendorizzate del repo stesso. Con `hallucination` aggiunto: `total_count 1`, e l'unico hit è casuale.
- `sign|signature|tamper|hash|append-only|immutable|WORM` cercati in `docs/tools/trajectory.md`, `docs/automation/tasks.md`, `docs/gateway/security/index.md` -> **nessun risultato pertinente**. È un'assenza di documentazione, non una prova di assenza nel codice.
- `SECURITY.md`, indicato da `THREAT-MODEL-ATLAS.md:28` come «current source of truth for vulnerability-report scope», **non è presente nel pacchetto npm**: l'elenco completo degli out-of-scope non è verificabile localmente.
- `/context list` **da CLI** non viene intercettato: finisce al modello, che risponde a prosa inventata. Non esiste una superficie CLI che esegua lo slash command in modo deterministico.
- `openclaw mcp probe` **non eseguito**: tutti gli otto server sono `enabled: false` e abilitarne uno avrebbe modificato `~/openclaw/openclaw.json`.
- `openclaw security audit` **non eseguito**: avrebbe potuto attivare probe o scritture.

d.5 Cosa è stato verificato e cosa è stato solo letto

Affermazione	Livello di prova
Esistenza e forma di ogni chiave di config citata	Verificato — schema estratto dall'installazione
Fail-open di <code>before_agent_finalize</code> (sei modi)	Verificato nel codice compilato , riga per riga
<code>message_sending</code> cancel terminale	Verificato nel codice in due percorsi di consegna
<code>compat.supportsTemperature</code> è un flag morto	Verificato per assenza — argomento negativo, regge perché in questi bundle i nomi di proprietà non sono manglati
Default dei timeout MCP	Verificato : costanti nel codice e output di <code>mcp status --verbose</code>
Peso del prompt di sistema	Misurato dal <code>systemPromptReport</code> persistito
Contenuto dell'audit ledger	Misurato sul database reale
Esistenza di <code>llm-grounded</code>	Verificato con <code>openclaw plugins search</code>
Architettura interna di <code>llm-grounded</code>	Letto dai test del repo; <code>src/</code> non è pubblicato su GitHub
Codice di Lobster	Letto dal tarball npm, non installato né eseguito
Comportamento runtime della sandbox	Solo letto : la sandbox non è configurata qui
Issue GitHub	Verificato via <code>gh api</code> autenticato
Documentazione dei framework non-OpenClaw	Letto dalle doc ufficiali; nessun test eseguito

(e) NON VERIFICATO

Tutto ciò che non è stato possibile confermare. Nessuna di queste voci è un'ipotesi presentata come fatto.

Su OpenClaw

1. **Nessun turno reale è stato eseguito per provare il cancello.** Non è stato scritto né installato alcun plugin: non è stato osservato un `cancel` o una `revise` dal vivo. Tutte le affermazioni sugli hook sono lette dal codice compilato e dai doc.
2. **Comportamento su harness non-embedded.** Il fail-open di `before_agent_finalize` è verificato sul percorso `embedded` (`embedded-agent-DGUuxGR2.js` / `selection-JInn13lc.js`). Il relay nativo `Stop` -> `before_agent_finalize` esiste (`dist/native-hook-relay-X0wljREM.js:808`), ma non è stato tracciato se applichi lo stesso tetto di 3.
3. `turnId` in `PluginHookBeforeAgentFinalizeEvent` (`dist/hook-types-DQ9eTy2x.d.ts:509`) è dichiarato nel tipo ma non appare popolato nella costruzione dell'evento a `selection-JInn13lc.js:13339-13351`. Probabilmente sempre `undefined` sul percorso `embedded`; non verificato altrove.
4. `retryCandidates` è usato dal merge multi-plugin (`hook-runner-global-BmIrGllG.js:356-368`) e letto da `lifecycle-hook-helpers-BwL6869q.js:166-169`, ma **non è nel tipo pubblico** né nei doc. Non va trattato come API.
5. **Ordine esatto di applicazione dei tre cancelli MCP in un singolo turno.** Verificata l'esistenza e la logica di ciascuno; non è stata tracciata una chiamata end-to-end che li attraversi tutti. La conclusione «AND, nessuno vince» è dedotta dalla struttura.
6. **Precedenza fra `toolsBySender` e le policy di `sandbox/subagent`.** La pipeline di default si ferma a otto step con `toolsBySender` ultimo, ma `sandboxPolicy`, `subagentPolicy` e `inheritedToolPolicy` compaiono anche in `dist/tool-dispatch-CEP-8k2K.js:66-77` e vengono applicate altrove. L'ordine `sandbox-vs-sender` **non è confermato**.
7. **Valori ammessi di `sandbox.backend`.** `ssh` e `openshell` sono documentati ma **non validati dallo schema** (stringa libera). I factory sono registrati dinamicamente via `Symbol.for("openclaw.sandboxBackendFactories")`: l'insieme reale dipende dai plugin caricati. Un `backend` scritto male probabilmente non viene rifiutato.
8. **Comportamento runtime della `sandbox`:** non osservabile, non è configurata su questa macchina.
9. **Durabilità del writer dell'audit ledger.** `docs/cli/audit.md:21` lo descrive come «bounded background writer». Cosa succede sotto pressione o al crash non è documentato, e il codice non è stato letto.
10. **Protezioni applicative su `audit_events`.** Nessun trigger SQL è presente nello schema della tabella — questo è un fatto. Che non esistano protezioni a livello applicativo è un'**inferenza**.
11. **Formato completo di `config-audit.jsonl`.** Letta solo la prima riga. Non è noto quali altri tipi di event esistano oltre `config.write`, né se la catena di hash venga verificata all'avvio. `docs/refactor/database-first.md:1031-1033` indica che il formato è in migrazione verso il DB.
12. **Costo in dollari nel bundle `trajectory`.** `artifacts.json` è documentato come contenente «usage» e «prompt cache»; il costo in USD non è elencato. Nessun bundle reale è stato aperto.
13. **Precedenza fra `captureContent.enabled: false` e una sottochiave `true`.** Non documentata in modo esplicito.
14. **Tabelle `capture_events` / `capture_sessions` / `capture_blobs`** presenti nel `sqlite`: i nomi suggeriscono una cattura di contenuto, ma non esiste documentazione corrispondente e lo schema non è stato ispezionato. Meritano un giro dedicato.
15. `~/./openclaw/logs/stability/` (vuota) e `~/./openclaw/internal-agent-runs/`: non indagate.
16. **Se `openclaw audit` legga sempre e solo dal `sqlite` locale**, o se con `--url` interroghi un Gateway remoto: non testato.
17. **Che `temperature` arrivi effettivamente sul filo.** La catena è tracciata in tre punti del codice, ma **nessun body reale è stato catturato**: servirebbe `openclaw proxy` con una richiesta viva, cioè una chiamata a pagamento. La procedura è in §6.4.

18. `compat.supportsTemperature` **mai letto** è verificato per assenza di occorrenze in tutta `dist` e in `node_modules/@openclaw/ai`. Non è escludibile un accesso dinamico via stringa costruita, né un consumo da parte di plugin provider esterni non ispezionati.
19. **Comportamento di `strict` sui provider non-OpenAI in produzione**: poiché nessun modello dichiara `supportsStrictMode: true` qui, il ramo «emette `strict: false` esplicito» non è mai attivo. Non provato con config modificata.
20. **Se `response_format` sia effettivamente onorato dai provider configurati**. OpenClaw fa pass-through senza validazione; un provider che lo ignora non genera errore né warning.
21. **Overhead reale di latenza MCP stdio vs plugin in-process**: non misurato.
22. **Il costo in tempo del thinking**: nessuna cifra, in nessuna forma, in `docs/tools/thinking.md`, `docs/providers/deepseek.md`, `docs/reference/prompt-caching.md`, `docs/tools/tool-search.md`, `docs/reference/code-mode.md`, `docs/concepts/context.md`. Le due sole menzioni di «latency» sono qualitative.
23. **Riduzione di round-trip di Tool Search**: il doc non dà nessun numero. L'unica affermazione misurabile è ordinale.
24. **Comportamento runtime di Task Flow**: verificato solo su doc e schema, mai eseguito.
25. **Se il retry replay-safe di Codex rilevi davvero tutti gli effetti collaterali**: dedotto dal testo, non testato.
26. **La percentuale 68% / Alpha** viene da `docs/maturity/scorecard.md:22-32`, pagina generata. L'evidenza QA sottostante non è stata verificata.
27. **Discrepanza di naming su Lobster**: il doc dice `@clawdbot/lobster` come runner embedded (`docs/tools/lobster.md:72`), il catalogo dei plugin esterni dice `@openclaw/lobster`. La discrepanza non è stata risolta.
28. **Non enumerati tutti i backend CLI per `nativeToolMode`**: quelli `non-always-on` non dichiarano il campo, quindi la loro presenza è dedotta dal fallback `undefined`.
29. **Numero di `supportsStrictMode: false` in `openclaw.json`**: 28 righe contate con `grep`, ma a quali id di modello corrispondano non è stato verificato.

Sulle issue GitHub

30. **I totali dei termini larghi** (`hook` 656, `lobster` 627, `sandbox` 425, `mcp` 243, `llm-task` 110) sono match full-text, non conteggi tematici. Sono state ispezionate solo le prime quindici per termine: **la coda non è stata guardata**.
31. **Nessun controllo oltre `is:open`**: non verificato se qualcuna sia già risolta da una PR non ancora mergiata, né lo stato di milestone o assegnatari.
32. **Storico del changelog non verificato**: GitHub Releases non è stato consultato.
33. **Alcune issue potrebbero essere già corrette upstream**: l'installazione locale è circa un mese dietro HEAD.

Sui plugin di terze parti

34. **`llm-grounded`: il sorgente non è stato letto**. La cartella `src/` è dichiarata in `package.json.files` ma **non è committata su GitHub**. La lista degli otto hook viene dal test che asserisce la registrazione (`tests/manifest.test.mjs`), non dalla lettura di `register()`. **Per un uso aziendale il pacchetto ClawHub va scaricato e il sorgente letto prima dell'adozione**.
35. **Discrepanza di versione di `llm-grounded`**: ClawHub elenca `v0.2.4`, il repo è a `0.3.5`. Quale versione installi effettivamente `openclaw plugins install clawhub:llm-grounded` non è stato verificato.
36. **Nessun conteggio di download o installazioni è esposto da ClawHub**: solo un intero «popularity» non spiegato (`llm-grounded` 14, `critiqor` 43).
37. **Nessuno dei plugin di terze parti è stato installato o eseguito**.

Sui framework alternativi

- 38. **Nome esatto della classe di errore di non-determinismo di Temporal:** la doc parla di «non-deterministic error», il simbolo non è confermato.
- 39. **Visibilità dei payload da parte del personale Temporal su Cloud:** la pagina Data Converter non lo afferma; la cifratura è opt-in via codec, non default. Nessun claim di cifratura end-to-end trovato.
- 40. **Rilevamento di divergenza in LangGraph:** nessun meccanismo trovato. Assenza di documentazione, non prova di assenza.
- 41. **Immutabilità o completezza garantita delle tracce LangSmith:** non dichiarata.
- 42. **BAML:** docs.boundaryml.com si autodichiara **vo**, «the legacy DSL»; BAML v1 è documentato altrove. Tutte le conclusioni su BAML valgono per vo.
- 43. **Se un fallimento di @assert in BAML inneschi un re-prompt:** la pagina checks-and-asserts non tratta retry.
- 44. **deps_type di Pydantic AI come meccanismo di access control:** la doc non lo presenta così. Quell'inquadramento non è verificato.
- 45. **DSPy: determinismo runtime, replay, audit, access control:** nessuna affermazione su nessuna pagina recuperata.
- 46. **Architettura "niente agente":** non ha documentazione di vendor perché non è un prodotto. Le proprietà attribuite derivano dalla struttura, non da una fonte citabile. La classe di guasto residua identificata (parafrasi infedele) e la sua mitigazione sono ragionamento, non citazione.

Sul metodo di questo documento

- 47. **Sette delle otto indagini parallele sono morte** all'uscita del processo e sono state rilanciate. Una di esse aveva annotato numeri di issue non verificati prima di essere corretta; le query sono state rieseguite integralmente e i numeri sono confermati, ma l'episodio è dichiarato in §d.1.
- 48. **Un turno reale a pagamento** è stato speso durante la ricerca sulla latenza, per un tentativo di `/context list` che non era intercettato. I numeri di §5.1 non provengono da lì.
- 49. **Alcune citazioni riga per riga di** `exec-approvals*.md`, `permission-modes.md`, `elevated.md`, `config-tools.md`, `multi-agent-sandbox-tools.md`, `operator-scopes.md` provengono da lettura delegata con citazione `file:riga`, e non sono state ricontrollate una per una in questa sessione.

Documento prodotto il 19 agosto 2026 su OpenClaw 2026.7.1-2 (0790d9f). Ogni affermazione non contrassegnata come NON VERIFICATO è ancorata a un percorso `file:riga` del pacchetto installato, a una pagina di documentazione presente localmente, o a un output di comando riportato letteralmente in §d.2.